slop v3: a computer made of stores — architecture and implementation design

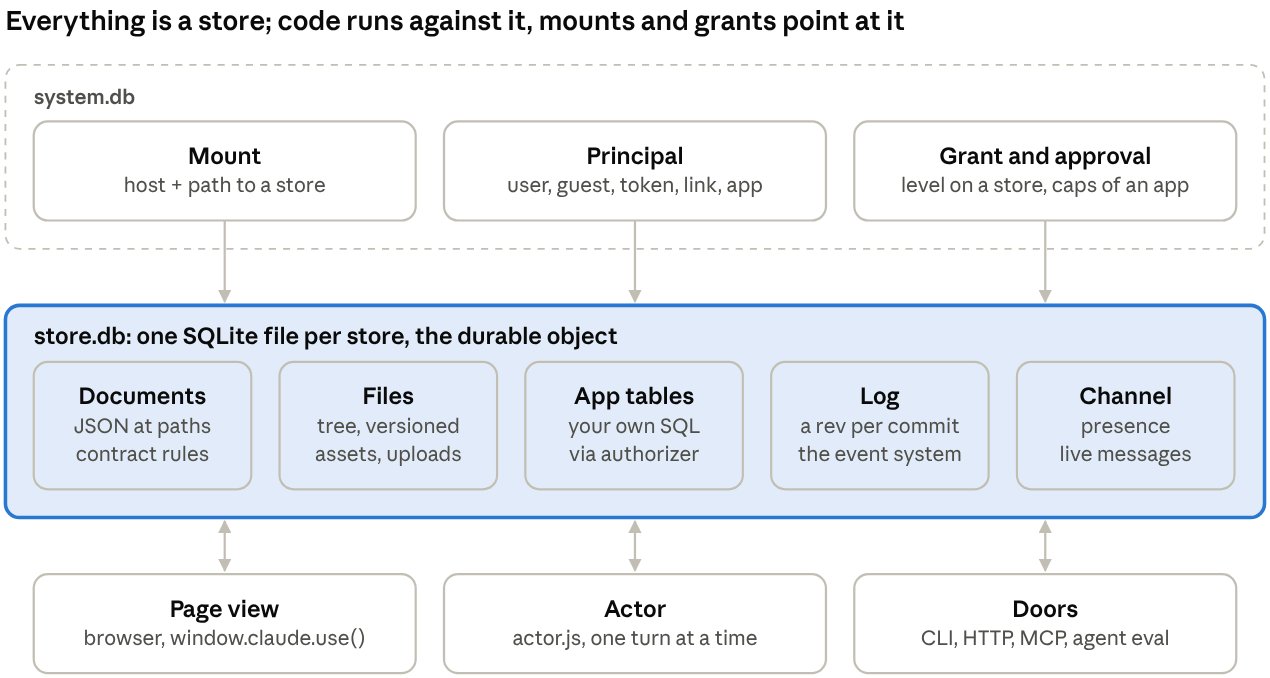
Oct 31, 2026 · @Manuel

1. Summary

v3 is built on one durable noun, the **store**: a SQLite file holding JSON documents, files, the app's own SQL tables, a log of every change, and a live channel. Everything else either is a store, runs against one, or points at one.

- An **app** is a store whose files are code.
- Its server half, `actor.js`, runs as the store's **actor**: one at a time, in a JS sandbox.
- Its pages run in the browser against the same store, through the same `use()` capabilities.
- A **mount** puts an app on a store at a URL.
- **Grants** decide who may do what.
- Every change is a revision in some store's log, and that log is the event system.

This is the Durable Object model with JS as the only language and SQL exposed to code.



v3 at a glance · system.db, the store, three runtimes

Read it top to bottom. `system.db` says who and where, the store holds the state, and three kinds of runtime work against it under one `can()`.

Decisions already taken

- **SQLite everywhere:** one file per store, plus one `system.db` for identity, grants, mounts and schedules.
- **SQL is exposed to code.**
  - An actor gets synchronous SQL over its own store, except the system tables.
  - Pages query the tables and views the manifest exposes, through a structured API; people and agents with admin rights get raw SQL.
- **No backwards compatibility** with slop v2 data, APIs or CLI. The one compatibility kept is the claude.ai artifact runtime contract (`window.claude.use`), because the apps we want to host (Claude Computer, Blocks, Pinup) are written against it.
- **Built twice.** First a browser workbench, which is the executable specification and the place to study the system. Then a Go foundation that passes the same conformance suite and adds server-side sandboxing.

Goals

1. One noun for state. Apps, databases, projects, agent conversations and the system itself are all stores.

2. One capability catalog, `use(name)`, in pages, actors and doors (CLI, HTTP, MCP), checked by one `can()`.
3. Host app builders unchanged, and let them make apps: content in a store can become an app at a URL in one step.
4. A UI whose top level is only Data, Apps, Sites, People and Activity. Everything else is a tab on an object, or hidden.
5. A kernel small enough for an agent to read whole: under 15,000 lines in each implementation, not counting the shell.

Non-goals for v3.0

- More than one server process, and replication.
- Billing, plans and quotas beyond per-store limits.
- Hostile multi-tenant isolation at internet scale. Process isolation is designed (section 15) but phased.
- A CRDT or text-merge engine in the kernel: apps bring their own, as Blocks brings Hypha.
- ES module syntax in actors.

Designed here, built in v3.1:

- actor-host processes and VM isolation (section 15.4, levels 2 and 3);
- fork-on-open;
- delegates that create further delegates;
- durable channel topics;
- slots other than `surface` and `opener`;
- `export --html`;
- app actions as MCP tools;
- reverting changes to SQL tables.

Nothing in v3.0 may block them.

2. Lineage: what each source gives v3

v3 takes the runtime contract from francois, the kernel and access work from slop v2, the server half from go-go-host and go-go-goja, and the shape of a self-modifying UI from Ultimate and go-go-os. What it drops is everything that existed only to sell copies or to make up for the missing server code.

Source	Keep	Drop
slop-computer v2	Versions as file trees, refs, route log and undo, safe mode `/_/`, `pkg/rt` (rules, queries, diff sync, conformance pages), guests, share links, site sign-in handoff, `comments` `wait`	`page` / `data` kinds, `app` column, target and instance as nouns, route columns (`bindings`, `record`, `page_from`, `live`, `access`, `create_policy`), the separate `events` table, nine access mechanisms
francois	The `window.claude.use` contract runtime, one SSE stream plus one RPC endpoint per view, `data/users/<id>` privacy, content-addressed blobs, device-code login, the model `Backend` interface with a fake	Billing and plans, launch codes and runtime grants, single-file releases, Go-template storefront, coarse `db_changed` re-queries
go-go-host	One SQLite file per site, dry-run then smoke then swap activation, host-to-runtime dispatch, narrowed short-lived agent deploy tokens	Orgs and memberships, capability toggles that the runtime ignores, `app.static` serving any host directory, a JS-configurable DB guard
go-go-goja	`engine` factory and `runtimeowner` (one goroutine owns a VM), `hotreload` blue/green swap, routes whose auth is checked before JS runs (`RoutePlan`, `Actor`, `GrantSet`), `repldb` leases with fencing, providers that package modules	The hard-coded `appauth.Authorizer` switch, raw DSN-configurable `database`

Source	Keep	Drop
go-go-os	APP-29′s object, harness and runtime split; VMs that return a widget tree for the host to draw; reflection documents that tell agents what an app can do; hydrate-then-stream timelines; approval requests (plz-confirm)	Apps compiled into the host as React modules, one Redux store as the database, a widget DSL that tries to cover everything
Ultimate and Pi Durable	Events committed with the change they describe, reactor cursors committed with the reactor′s writes, forks as of a point, surfaces with slots and a docked agent, hot reload that falls back to the last working code, a small agent-readable `api` per extension	Nothing: they inform the design rather than donate code
Ramon′s apps (Claude Computer, Blocks, Meta Blocks, Pinup, Librarian)	The requirements: in-page jobs that should run with the page closed, nested apps that want URLs, MCP connectors, agents as peers in a log, Librarian′s three doors into one implementation	Nothing: they are the first tenants
Form Skate design systems and the textbook handbook	The workbench shell (dock of tiles, Kiln style), the one-tool `eval` agent with Auto, Ask and Read-only modes, figures and the executable-textbook method	Nothing: they are the look and the method of the workbench

The essay "Software is just content" supplies the test the design must pass. Making an app should feel like writing content. The durable value is the plain infrastructure under it: one log, one language, unnoticed.

3. Vocabulary

The kernel has five nouns: store, app, mount, principal and grant. The rest of this list names parts of them or moments in their life. Each word means one thing everywhere: in code, UI, CLI and docs.

Term	Meaning	Lives in
Store	The durable object: an id, a type, a default app, and one SQLite file holding documents, files, app tables, a log and alarms	`stores/<id>/store.db`
Document	A JSON object at an even-segment path (`tasks/t1`), with the contract's semantics	`_docs`
File	Bytes at any path, either in the versioned **tree** zone or the runtime **assets** zone	`_files` plus the blob store
App table	Any table an app creates with its migrations	the store file, names without a leading `_`
Turn	One run-to-completion slice of work against a store: an actor handling one event, one page write, one door verb	in memory
Commit	What a turn writes, atomically. It gets the next **rev** (a per-store integer)	`_log` , `_changes`
Step	A group of commits, possibly across stores, made by one principal for one purpose (an agent step, a push). It can be kept or reverted	`steps` in `system.db`
Snapshot	The tree zone frozen at a rev, numbered `n` , optionally with a full database image	`_snapshots` , `_snapshot_files`
Ref	A movable name for a snapshot: `main` , `take/denser`	`_refs`

Term	Meaning	Lives in
Fork	A new store copied from a snapshot image or from now	new store
App	A store whose tree contains `app.json`. Its code runs against any store whose `app` is it	the app's store
Actor	The live server instance of one store, running its app's `actor.js`	memory: QuickJS (workbench) or goja (Go)
View	One open page: one tab, one app version, one store, one principal	memory; a secret, a stream, an RPC endpoint
Door	A way in for people and agents: CLI, HTTP or MCP, all generated from one verb table	kernel
Mount	Host plus path pattern pointing at a store (fixed, named by the path, or created); the store's app and version decide the code	`mounts` in `system.db`
Principal	Who acts: a user, guest, agent token, share link, app, delegate or the system	`system.db`
Level	`view` < `interact` < `admin` < `owner`, a principal's standing on a store	computed
Grant	Gives a subject a level on a store, or an `instantiate` right on an app	`grants` in `system.db`
Capability	A named API from `use(name)`: `db`, `sql`, `files`, `room`, `sample` and so on	app manifest, approvals
Approval	An admin's yes to the sensitive capabilities of one app version	`approvals` in `system.db`
Channel	A store's live presence and messages (the contract's `room`)	memory, or the log for durable topics
Reactor	An actor handler subscribed to a log, with a cursor	`_cursors`

Term	Meaning	Lives in
Alarm	A timed wake-up for a store's actor, one-off or recurring	`_alarms`, mirrored to `schedules`
Surface	A unit of UI in the shell: a full iframe app, or a declarative widget tree	the shell
Slot	A named place in the shell that apps can fill: a surface, a panel, an opener for a store type	the shell

4. The store

A store is one SQLite file that holds everything about one durable thing: its documents, files, app tables and history. It can therefore be copied, forked, exported, quota-limited and deleted as a unit. Identity, grants and mounts live outside it, in `system.db`, so that access can be decided without opening every store.

4.1 On disk

```
<root>/
  system.db                       registry, principals, grants, mounts,
approvals, schedules, steps
  stores/<id>/store.db            one file per store (WAL in Go; rollback
journal in the browser)
  stores/<id>/images/<n>.db       optional full images taken at snapshot n
(VACUUM INTO)
  blobs/sha256/<ab>/<cd>/<hash>   file bytes, content-addressed, shared by
every store
```

- **Store ids** are lowercase slugs, `[a-z0-9][a-z0-9-]{0,62}` (`journal`, `ws-team`, `blocks`). An unnamed store gets `s-` plus eight base32 characters. The id is the address everywhere: CLI, URLs, `use('stores').open(id)`.
- **The system store is `system.db` itself.** Besides the registry tables, it has the same `_log`, `_changes` and `_docs` tables as any store. So a grant, mount or approval and its log entry commit in one transaction, and the Activity surface reads the system log like any other log.

4.2 System tables inside a store

Every table whose name starts with `_` belongs to the kernel. App code can read some of them through views (section 5) and can write none. The DDL is in Appendix A; the shape is:

Table	One row per	Key columns
`_meta`	setting	`key`, `value`: id, type, app, app_ref, app_version, last_good, owner, created_at, epoch, lease_epoch, limits
`_docs`	document	`path`, `coll`, `body` (JSON, NULL = tombstone), `version`, `rev`, `size`, `by`, `at`
`_files`	file	`zone` (`tree` or `assets`), `path`, `sha256`, `size`, `type`, `rev`, `by`, `at`
`_log`	commit	`rev`, `at`, `principal`, `via`, `app`, `step`, `summary`
`_changes`	change inside a commit	`rev`, `seq`, `kind`, `key`, `op`, `version`, `old`, `new`
`_snapshots`	snapshot	`n`, `rev`, `tree`, `image`, `message`, `by`, `at`, `parent`
`_snapshot_files`	file in a snapshot	`n`, `path`, `sha256`, `size`, `type`
`_refs`	ref	`name`, `n`, `at`, `by`
`_migrations`	applied migration	`app`, `name`, `sha256`, `rev`
`_cursors`	reactor cursor	`name`, `source`, `epoch`, `rev`
`_alarms`	alarm	`name`, `at`, `every`, `data`, `running`
`_leases`	contract lease	`path`, `holder`, `until`, `data`

The last four tables are bookkeeping. Their writes ride along in a commit but never make one alone, and they never appear in `_changes`.

4.3 Documents

Documents keep the claude.ai contract's semantics exactly, because hosted pages depend on them.

- **Paths alternate** collection and document. A path with an even number of segments names a document; an odd number names a collection. At most 16 segments, 200 bytes each, 1,000 bytes in all.
- **Bodies are JSON objects**, at most 256 KiB and 32 levels deep. A store holds at most 25,000 live documents.
- **Operations** are `set`, `update` (deep merge for objects, arrays replaced), `delete`, `acquire` (lease) and queries with up to ten `where` filters, `orderBy` and `limit`.
- **Versions:** `version` counts changes to one document; `rev` is the store revision that last wrote it. A delete leaves a tombstone for seven days, so a view that reconnects learns of it.
- **Queries are SQL** over `_docs` with `json_extract`. A manifest can ask for expression indexes on a collection's fields (section 6.1); the kernel creates them as `_idx_<coll>_<field>`.

4.4 Files and blobs

- **Two zones.** The **tree** zone is the versioned content: code, pages, records, Markdown posts, images that belong to the content. The **assets** zone holds what people upload while using an app (the contract's `assets`). Snapshots include only the tree zone.
- **Bytes live in the shared blob store**, keyed by SHA-256; `_files` holds only metadata.
  - `system.db` keeps a reference count per blob (`blob_refs`). The kernel raises and lowers it as files, snapshots, images and step history refer to the blob.
  - A blob at zero for 7 days is deleted. No scan of store files is ever needed.
- **Records:** files in the tree zone named `*.md` (with front matter), `*.json` or `*.csv` can be read as records (v2's data artifacts). They are content, not a separate kind of store.

4.5 The log, commits and changes

Every write goes through a turn, and a turn that changes anything ends in one commit, with one new rev.

- **`_log`** has one row per commit: who (`principal`, always a user, guest, app or `system`), through what (`via`: `page`, `actor`, `reactor:<name>`, `cli`, `http`, `mcp`, `kernel`, plus the token id when an agent acted), which app version, which step, and a one-line

summary.

- **`_changes`** has one row per changed thing in that commit.
  - `kind` is `doc`, `file`, `table`, `row`, `ref`, `snapshot`, `schema` or `msg`. `msg` is for durable channel topics, in v3.1.
  - `key` is a path, a table name, `table:rowid` or a ref name.
  - Documents record their new `version`. The old body goes into `old` only when the commit belongs to a step, because only steps can be reverted.
  - Files record old and new hashes.
- **What makes a commit:** a change to documents, files, app tables, refs, snapshots or schema. Bookkeeping writes (cursors, alarms, leases, migrations records) never make one on their own.
- **Retention** ends at whichever limit is hit first: the newest 100,000 changes, 32 MiB of `_changes`, or 7 days.
  - Tombstones are kept as long.
  - A cursor older than retention gets `reset: true` and must reload.
  - Steps can be reverted only while their changes are still retained.
- **Ordering and restores:** revs are dense and strictly increasing per store. Restoring an image changes `_meta.epoch`, and every cursor carries `(epoch, rev)`, so after a restore cursors reset instead of silently skipping commits. There is no global order across stores; the system log orders platform events.

4.6 Snapshots, refs and forks

- **A snapshot** freezes the tree zone at the current rev: `_snapshot_files` gets the path-to-hash list, and `tree` is the hash of that list in canonical order. Pushing code to an app is "write the tree zone, then snapshot" in one turn.
- **An image** is `VACUUM INTO images/<n>.db`, a full copy of the database.
  - `VACUUM INTO` cannot run inside a transaction, so images are taken **between turns**, on a kernel connection whose `ATTACHED` limit is raised to 1. The authorizer still denies `ATTACH` to code.
  - `files.snapshot({ image: true })` from an actor returns `n` at once, and the image follows right after the turn.
  - Images are taken on demand, before every activation, and daily (keep 7).

- **Refs** point at snapshots. `main` moves on every push unless the push names another ref. A store follows its app's ref (`_meta.app_ref`) or a pinned version. Moving a ref is a change of kind `ref` , so it shows in the log.
- **A fork** creates a new store from one of three sources:
  - a snapshot's image: full state;
  - its tree only: code and content, empty data;
  - now: an image taken between turns.

  The new store records `forked_from` and `forked_at` in `_meta` .

4.7 Export and import

- **As a file:** `slop export <store> --sqlite` writes `store.db` plus the blobs it references, as a zip. This is the backup format and the Durable-Object-style move between machines.
- **As a folder:** `--folder` writes `store.json` (meta), `tree/` (the tree zone as plain files), `docs.jsonl` , `tables/<name>.jsonl` with `schema.sql` , and optionally `log.jsonl` . This is the format people and agents edit, diff and commit to git.
- **Import** takes either form and creates a new store; it never merges into an existing one.

4.8 Limits

Limits are per store, stored in `_meta` , with defaults an admin can raise.

Limit	Default	Enforced by
Database size	256 MiB	`PRAGMA max_page_count`
Live documents	25,000	kernel, error `quota_exceeded`
Document body	256 KiB, depth 32	kernel
File size	20 MiB (assets), 50 MiB (tree)	kernel
Tree size	200 MiB, 5,000 files	kernel
Actor CPU per turn	250 ms	JS interrupt handler
Activation (migrations, setup, smoke)	5 s	the transaction rolls back

Limit	Default	Enforced by
SQL time per statement from pages and agents	250 ms	progress handler or context deadline
Rows per query result	10,000	kernel
Change history	100,000 changes, 32 MiB, 7 days: first limit wins	kernel

5. SQL for apps

Every store is a real SQLite database, and code gets SQL on it at three strengths.

- **The actor,** the app's own backend, runs any SQL on its app tables.
- **People and agents with `admin`** on a store get raw SQL through the doors. They can read everything; with `--write` they can change app tables.
- **Pages** never send SQL text. They query the tables and views the manifest exposes, through a structured API, and the kernel writes the SQL.

SQLite's authorizer keeps all three away from kernel tables, attached files and extensions. Row privacy comes from views that call a function returning the current principal.

Why pages get no raw SQL. The authorizer's fifth argument names only the innermost view or trigger, and a CTE can borrow any view's name. The review reproduced this on SQLite 3.53. So "allow reads that come through an exposed view" cannot be enforced on SQL a page writes. Kernel-written SQL never defines CTEs, so for it the rule holds.

5.1 Three callers, three powers

Caller	API	Reads	Writes	DDL
Actor (`actor.js`)	`use('sql')`: `all`, `one`, `run`, `transaction`; synchronous	app tables; `slop_docs`, `slop_files`, `slop_log`	app tables, inside the turn's transaction	migrations only

Caller	API	Reads	Writes	DDL
Page	`claude.use('slop')`, then `.sql.select(name, query)` and `.sql.watch(name, query, onRows)`	exposed tables and views, at their read level	none: pages write through actor actions or `db` documents	none
Door, admin only	`slop sql query <store> "..."`, `slop sql exec <store> "..." --write`	everything in the store, kernel tables included, on a read-only connection	app tables, with `--write`	none: DDL ships as migrations

The structured query a page sends:

```
const slop = await claude.use('slop');
const rows = await slop.sql.select('my_orders', {
  columns: ['id', 'item', 'qty'],
  where: [['qty', '>', 2], ['item', 'like', 'tea%']],   // at most 10
filters: = != < <= > >= like in is
  orderBy: [['at', 'desc']], limit: 50, offset: 0 });
const stop = slop.sql.watch('my_orders', { orderBy: [['at', 'desc']], limit:
50 }, rows => render(rows));
```

The kernel builds the SQL:

- It checks that `name` is exposed at the caller's level.
- It checks every column against `PRAGMA table_info` of that table or view.
- It binds every value as a parameter and prepares `SELECT <columns> FROM "<name>" WHERE ... ORDER BY ... LIMIT ?`.

5.2 Names and connection functions

- Names starting with `_` are the kernel's (section 4.2). Names starting with `slop_` are reserved. Everything else belongs to the app.
- Three read-only kernel views serve actors and admins:
  - `slop_docs(path, coll, body, version, rev, at)`;
  - `slop_files(path, zone, sha256, size, type, rev)`;

    - `slop_log(rev, at, principal, via, summary)`.
- These functions are bound to the acting principal before each statement:
    - `slop_principal()` returns:
        - the acting user's id (`usr_...` or `vis_...`), including when an agent token or delegate acts for that user (the token shows in the log's `via`);
        - `app:<id>` for an actor acting as itself;
        - `anon` on public HTTP routes.
    - `slop_level()`, `slop_can_read(path)`, `slop_can_write(path)` and `slop_now()` give the level, the document rules and the turn's time.
- **Connections:** the actor owns the store's one writer connection; pages and doors use reader connections. `TEMP` objects are denied everywhere, so nothing made on one connection can hide a name on another.

5.3 Migrations

- An app ships migrations as `sql/NNN_name.sql`. They are applied in order and recorded in `_migrations`.
- **One transaction.** Activation (section 6.4) runs the pending migrations, then `setup`, then the optional `smoke` action, in one transaction, and rolls all of it back if any step fails. SQLite DDL is transactional, so no copy of the store is needed. The whole transaction gets 5 seconds.
- **Rollback is a restore.** An image is taken just before, between turns (section 4.6). There are no down migrations.
- **Allowed:** tables, indexes, views and triggers with app names, and FTS5 virtual tables (`CREATE VIRTUAL TABLE ... USING fts5`; FTS5 is the only module allowed).
- **Not allowed:** `_` or `slop_` names, `ATTACH`, extensions, other virtual-table modules.

5.4 The authorizer policy

One authorizer callback per connection checks each statement while it is prepared. Before preparing, the kernel sets the caller kind: `actor`, `migration`, `page` (kernel-written), `admin` (door) or `kernel`.

Action code	Actor	Migration	Page (kernel-written)	Admin door
`SELECT`; `READ` on app tables and views	allow	allow	allow: only exposed names can appear, because the kernel wrote the SQL	allow
`READ` on `_*`	allow: the actor owns its store	deny	deny	allow
`READ` on `sqlite_schema`	allow	allow	deny	allow
`INSERT`, `UPDATE`, `DELETE` on app tables	allow	allow	deny	with `--write`
`INSERT`, `UPDATE`, `DELETE` on `_*` or `sqlite_*`	deny	deny	deny	deny
`CREATE`, `DROP`, `ALTER` of tables, indexes, views, triggers	deny	allow on app names	deny	deny
`CREATE_VTABLE`, `DROP_VTABLE`	deny	`fts5` only	deny	deny
Any `TEMP` object	deny	deny	deny	deny
`ATTACH`, `DETACH`	deny	deny	deny	deny

Action code	Actor	Migration	Page (kernel-written)	Admin door
`PRAGMA`	`table_info`, `table_xinfo`, `index_list`, `index_info`, `foreign_key_list`, `data_version`	the same, plus `user_version`	none	the same as actor
`FUNCTION`	allowlist: SQLite core, JSON, math, FTS5 auxiliaries, `slop_*`	same	same	same
`TRANSACTION`, `SAVEPOINT`	deny; use `sql.transaction(fn)`	deny	deny	deny
`ANALYZE`, `REINDEX`	deny	allow	deny	deny
`RECURSIVE` (CTEs)	allow	allow	none	allow; time-bounded

- **FTS5 runs its own statements** on its shadow tables (`<name>_data`, `<name>_idx`, …) and runs `PRAGMA data_version`. The policy maps a shadow table to its FTS5 table and allows `data_version`, so full-text search works for actors and migrations.
- **Connection settings** on every app connection:
  - `SQLITE_LIMIT_ATTACHED` = 0, `LENGTH` = 16 MiB, `SQL_LENGTH` = 100 KiB, `EXPR_DEPTH` = 100;
  - `load_extension` off; `trusted_schema = OFF`.

  Images are taken on a separate kernel connection where `ATTACHED` is 1 (section 4.6).
- **Time:** page and door statements stop after 250 ms (a progress handler in the workbench, a context deadline in Go). Actor statements share the turn's budget.

5.5 Exposing tables and views

```
{ "slop": { "sql": { "expose": {
  "products":     { "read": "view" },
  "my_orders":    { "read": "interact" },
  "sales_by_day": { "read": "admin" }
} } } }
```

Row privacy is a view written in the app's migrations. Writes are actions:

```
CREATE TABLE orders (id INTEGER PRIMARY KEY, owner TEXT NOT NULL, item TEXT,
qty INTEGER, at INTEGER);
CREATE VIEW my_orders AS SELECT id, item, qty, at FROM orders WHERE owner =
slop_principal();
```

```
actions: {
  order(ctx, { item, qty }) {
    ctx.need('interact');
    return sql.run('INSERT INTO orders(owner, item, qty, at) VALUES
(?,?,?,?)', ctx.principal.id, item, qty, ctx.now);
  },
}
```

Exposing a table exposes every row in it at that level; to narrow it, expose a view instead. Validation and ownership live in one place, the actor.

5.6 Changes from SQL

SQL writes appear in the log like document writes, so reactors, views and agents see them the same way.

- **Table changes, always.** The authorizer's write set names every table a statement may write. That includes:
  - trigger targets;
  - `WITHOUT ROWID` tables;
  - FTS5 tables (mapped from their shadow tables);
  - `DELETE` without `WHERE`.

  A turn that wrote app tables records one `table` change per table. This is the change subscribers rely on.

- **Row hints, when SQLite gives them.** The update hook reports `(op, table, rowid)` for most row changes in rowid tables, trigger writes included. It misses:
  - `DELETE` without `WHERE`;
  - rows that `REPLACE` removes;
  - virtual tables.

  The kernel records what it gets as `row` changes so watchers can re-read less. Nothing relies on the hints being complete.
  - A hook may not write to the database, so the kernel buffers the hints and writes `_changes` at commit.
  - Prefer `INTEGER PRIMARY KEY` tables, because other rowids can change on `VACUUM`.
- **Schema changes** from migrations record kind `schema`, and watches re-prepare.
- The **preupdate hook** gives old row values, for `WITHOUT ROWID` tables too. It is in sqlite-wasm, and in mattn behind a build tag. It is the route to reverting SQL changes in v3.1.

5.7 Live queries

`slop.sql.watch(name, query, onRows)` in a page; in an actor, `on: { 'table:<name>': ... }`.

1. The kernel prepares the generated SQL and records its read set: every table the authorizer reports as read, including those behind views.
2. After each commit, it checks for `table` changes in that read set.
3. On a match, it re-runs the query and delivers all the rows, at most once every 100 ms per watch. Diffs are v3.1.

Limits: 32 watches per view; 250 ms per run, or the watch stops with `query_too_slow`; 10,000 rows.

5.8 Two kinds of data, one database

- **Documents** fit pages written for claude.ai, small apps and agents: no schema, path rules, mirrors that work offline.
- **App tables** fit data with shape, joins, aggregates and full-text search.
- **Both live in the same transaction.** An actor can move a document into a table, or index document fields in a table, in one turn. No sync layer sits between them.

6. Apps and the actor

The server file is `actor.js`, and it is the store's actor. It is the code of a Durable Object class; each store running the app is one instance. It handles one event at a time, every event runs as a turn, and each turn is one SQLite transaction. All of an app's server behaviour hangs off one call, `defineActor({...})`: actions the page calls, HTTP routes, reactions to log events, and alarms.

6.1 The manifest, `app.json`

A store is an app when its tree contains `app.json`. The `capabilities` block keeps the claude.ai contract's shape, so a page published to claude.ai runs here unchanged. Everything slop-specific sits under `slop`.

```
{
  "title": "Tracker",
  "page": "index.html",
  "actor": "actor.js",
  "opens": "tracker.board",
  "capabilities": {
    "db":   { "rules": [{ "path": "boards/*", "read": "view", "write":
"interact" }],
              "indexes": [{ "coll": "tasks", "field": "due" }] },
    "user": { "scopes": ["profile"] },
    "room": { "topics": { "cursor": "interact" } },
    "sample": {}
  },
  "slop": {
    "sql":      { "expose": { "my_tasks": { "read": "interact" } } },
    "fetch":    { "allow": ["https://api.github.com"] },
    "secrets":  ["GITHUB_TOKEN"],
    "spawn":    true,
    "platform": ["stores:create", "mounts:request"],
    "surfaces": [{ "id": "board", "title": "Board", "slot":
"opener:tracker.board", "render": "actor:board" }],
    "api": "API.md"
  }
}
```

- `page` is the HTML entry. It is optional: an app with only an actor serves HTTP and actions.

- `actor` is the server entry. It is optional: most claude.ai-style apps need none.
- `opens` is the store type this app runs on. New stores made with this app get this type, and "Open with" lists apps by the types they open.
- `slop.api` points at a Markdown file describing the actions. Agents read it (go-go-os reflection, Ultimate's `api.ts`), and the kernel checks that it names every action.
- **One app per store.** `_meta.app` names the app. `_meta.app_ref` (default `main`), or a pinned version, picks the code, and `_meta.app_version` records the version running now.
  - Mounts name only the store.
  - Another app works on the store through the owning app's actions, or reads it by grant from a store of its own.
  - A preview of another version runs on a fork, never on the live store.
- **Always available, no declaration needed:** `sql`, `files`, `log`, `alarm`, `time`, `ui`, `actions`, `stores.open`, and `claude.use('slop')` in pages. Everything else must be declared (Appendix C).

6.2 `actor.js`

```
const sql = use('sql'), docs = use('db'), room = use('room');

defineActor({
  // Runs once per (store, app version), in the activation transaction, after
migrations.
  setup(ctx) {
    if (!sql.one('SELECT 1 FROM boards LIMIT 1')) sql.run('INSERT INTO
boards(name) VALUES (?)', 'Inbox');
  },

  // Called from pages: (await claude.use('slop')).actions.addTask({...}).
Returns JSON.
  actions: {
    addTask(ctx, { board, title }) {
      ctx.need('interact');
      const { lastRowid } = sql.run(
        'INSERT INTO tasks(board, title, owner, at) VALUES (?,?,?,?)',
        board, title, ctx.principal.id, ctx.now);
      return { id: lastRowid };
    },
    async summarize(ctx, { board }) {
```

```
      ctx.need('interact');
      const rows = sql.all('SELECT title FROM tasks WHERE board = ?', board);
// turn 1
      const text = await use('sample').create({ prompt: rows.map(r =>
r.title).join('\n') });
      docs.doc(`summaries/${board}`).set({ text, at: ctx.now });
// turn 2, new ctx.now
      return { text };
    },
  },

  // HTTP on the store's mounts. Auth is decided before the handler runs;
'public' runs as principal anon.
  http: {
    'GET /api/tasks': { level: 'view', run: (ctx, req) => ({ json:
sql.all('SELECT * FROM my_tasks') }) },
    'POST /hooks/github': { level: 'public', run: (ctx, req) => { /* verify
signature, write */ return { status: 204 }; } },
  },

  // Reactions to log events (section 8). Synchronous: one turn each, exactly
once per commit.
  on: {
    'table:tasks'(ctx, commit) { room.emit('changed', { rev: commit.rev });
},
    'doc:inbox/*'(ctx, commit) { for (const c of commit.changes) { /* move
documents into tables */ } },
    'store:journal doc:posts/*'(ctx, commit) { /* another store's log, by
grant */ },
  },

  // Alarms: may be async. Recurring by cron here; one-off with
use('alarm').at(t, name, data).
  every: {
    nightly: { cron: '0 3 * * *', run(ctx) { sql.run('DELETE FROM tasks WHERE
done AND at < ?', ctx.now - 30 * 864e5); } },
  },

  // Declarative surfaces (section 11): return a widget tree.
  surfaces: {
```

```
    board(ctx, props) { return use('ui').table(sql.all('SELECT title, owner
FROM my_tasks')); },
  },
});
```

`ctx` carries what differs per event:

- `principal`: `{ id, kind, name, level }`. For an agent token or delegate, `id` is the user it acts for; the log's `via` names the token.
- `store`: `{ id, type }`; `app`: `{ id, version }`.
- `now`: when this turn started. Each turn after an `await` gets a new `now`.
- `rev`: the store's rev when the turn began.
- `step`, `event` and `req`.
- `need(level)`, which throws `permission_denied`; `exclusive(fn)`; `log(...)`.

Capabilities come from `use(name)`, which in an actor returns at once, because the manifest is known when the actor starts.

6.3 Turns are transactions

The actor's guarantees are Durable Objects' input and output gates, made concrete with SQLite.

1. **One event at a time.** The actor takes events from one inbox: action calls, HTTP requests, log events for reactors, channel messages, alarms. Nothing else runs while a turn runs.
2. **A turn is one transaction.** The kernel:
   a. opens `BEGIN IMMEDIATE`;
   b. calls the handler;
   c. runs the promise jobs it queued, so awaiting `db` or `sql` stays inside the turn;
   d. commits once nothing but real I/O is pending.

   The commit gets one rev, and its changes go to the log.
3. **After an `await` of real I/O, a new turn starts.** Real I/O means `fetch`, `sample`, another store's action or `time.sleep`.
   - The continuation runs in a fresh transaction, with its own `ctx.now`. Writes before the await stay committed.
   - Continuations live in memory. If the actor crashes, a pending one is lost and the action's caller gets `actor_error: lost`.

4. **Other events can run while a handler awaits.** `ctx.exclusive(async () => {...})` holds the inbox until its promise settles. It is DO's `blockConcurrencyWhile`.
5. **A throw rolls back that turn only.** The caller gets the error with code `actor_error`; earlier turns of the same handler stay committed.
6. **Output waits for the commit.** Responses, channel messages and outgoing events from a turn are released after its commit succeeds; if the commit fails, they are dropped.
7. **Reactors are synchronous.** A reactor's writes and its cursor move commit in the same turn; that is what makes it exactly once.
   - An `on` handler that returns a pending promise is an error. The linter catches it, and at run time the turn rolls back with `actor_error: async_reactor`.
   - To do I/O in reaction to a change, a reactor writes a task row and sets an alarm.
   - A reactor never receives commits made by its own turns.
8. **Alarms may be async.** The alarm row is marked `running` until the handler's last turn, which removes it. A crash in between makes the alarm fire again, so alarm work must be idempotent.
9. **Time is the turn's.** `ctx.now` and `Date.now()` return the turn's start time, so a turn is repeatable under the workbench's virtual clock.

6.4 Lifecycle and activation

State	Meaning	Leaves when
cold	no runtime in memory	an event arrives for the store
waking	load the app version's files, run `actor.js` top level, apply pending migrations, run `setup` if the version is new to this store	top level and setup succeed, or fail into `broken`
active	handling its inbox	60 s without events, or eviction under memory pressure
hibernated	runtime closed; alarms stay registered in `system.db` `schedules`	the next event or alarm
broken	3 crashes in 1 minute, or waking failed	a new app version, or an admin's restart

- **Activation** of a new app version on a store has three steps:

1. Take an image, between turns.
2. In one transaction (section 5.3), apply pending migrations, run `setup`, then run the optional `smoke(ctx)` action.
3. Commit and swap the runtime, like go-go-goja's `hotreload`. If any step fails, roll back, stay on the previous version, and show the reason in Activity.

- **When:**
  - Pushing to an app activates it on its own store at once, inside the push.
  - Other stores running the app activate on their next wake, at most 4 at a time.
- **Fallback:** if a new version goes `broken` within 10 minutes, what happens depends on its migrations.
  - If it added none, the store returns to `last_good` on its own. This is Ultimate's rule that the old code keeps running.
  - If it added migrations, the old code cannot run on the new schema. The store stays `broken` until a newer version arrives or an admin restores the image taken before activation.

6.5 The JS dialect

Actor code must run unchanged on QuickJS (workbench) and goja (Go), so it is the intersection of the two.

- **Language:** ES2020-level syntax: `let` / `const`, arrow functions, classes, destructuring, spread, template literals, generators, `async` / `await` (goja has had async/await since December 2022).
- **Modules:** no `import` / `export` syntax, because goja's ES module support is unfinished. `require('./lib.js')` loads files from the app's tree, CommonJS-style, and `defineActor` registers the entry.
- **Globals:** `use`, `defineActor`, `require`, `console`, `JSON`, `Math`, `Date`, `TextEncoder` / `TextDecoder`, `structuredClone`. There is no DOM, no Node built-ins, no `setTimeout` (use alarms or `await use('time').sleep(ms)`, which ends the turn), and no raw network.
- **Lint:** `slop lint` parses actor files as ES2020 scripts and rejects module syntax and unknown globals before a push is accepted.

6.6 Where code runs

Code	Runs in	Typical use
Page (`index.html` and its scripts)	the visitor's browser, on the store's own origin	the app's UI; contract capabilities
Actor (`actor.js`)	the server (QuickJS in the workbench, goja in Go), one per store	validation, SQL, secrets, network, schedules, reactions, webhooks
Surfaces	either: rendered by the actor, or by a sandboxed page script	panels and openers inside the shell
Spawned child	a nested view or actor with narrower capabilities	apps made inside apps (section 13)

7. Capabilities: one `use()` catalog

There is one list of capabilities, and each has the same name and the same rules wherever it is offered. Pages get the claude.ai contract's names from `window.claude.use(name)`, plus slop's extras in one namespace from `claude.use('slop')`. Actors call `use(name)` directly. Doors expose the same operations as verbs (section 12). A capability the manifest does not declare is `null`, as the contract requires.

7.1 The catalog

Name	Page	Actor	Door verbs	Declare	What it is
`db`	contract, async	same API; writes join the turn	`docs.*`	yes (contract)	documents (section 4.3)
`sql`	`slop.sql.select`, `slop.sql.watch` on exposed names	`use('sql')`, synchronous	`sql.query`, `sql.exec` (admin)	always on; pages need `slop.sql.expose`	app tables (section 5)
`files`	`slop.files` (read the tree)	`use('files')`, read and write	`files.*`	always on	tree and assets zones

Name	Page	Actor	Door verbs	Declare	What it is
`assets`	contract	through `files`	`files.* --zone assets`	yes (contract)	uploads, in the contract's shape
`room`	contract	`use('room')`: emit, peers; receive with `on: {'channel: <topic>'}`	`room.*`	yes (contract)	channel and presence
`user`	contract	`ctx.principal`, `use('people')`	`whoami`	contract: always allowed	who is here
`comments`	contract	`use('comments')`	`comments.*`	yes (contract)	threads anchored on views
`downloads`	contract	none	none	yes (contract)	save a file from the page
`artifact`	contract `publish(html)`	none	`files.push`	yes, plus platform `refs:write`	snapshot this app's own tree
`sample`	contract	`use('sample')`	none	yes, approval, per-user consent	model calls on server credentials
`mcp`	contract	`use('mcp')`	none	yes, approval per server	connectors the host runs
`permissions`	contract, built in	none	none	always on	consent prompts
`log`	`slop.log`	`use('log')`	`log.*`	always on	this store's log; others by grant
`actions`	`slop.actions`	none (call directly)	`actor.call`	always on	call the store's actor
`stores`	`slop.stores.open`	`use('stores')`	`store.*`	always on for open; platform `stores:create` to create	other stores, by grant

Name	Page	Actor	Door verbs	Declare	What it is
`spawn`	`slop.spawn`	`use('spawn')`	none	yes, approval	child views and handles with fewer capabilities
`alarm`	none	`use('alarm')`	`alarm.*`	always on	wake-ups
`fetch`	none	`use('fetch')`	none	yes, approval per origin	outbound HTTP to allowed origins
`secrets`	none	`use('secrets')`	`secret.set` (admin)	yes, approval per name	values admins set for this app
`platform`	`slop.platform`	`use('platform')`	the management verbs	yes, approval per permission	mounts, grants, refs, stores, questions
`ui`	none	`use('ui')`	none	always on	widget-tree builder for surfaces
`time`	none	`use('time')`	none	always on	`sleep(ms)`, which ends the turn; the clock

7.2 Signatures of the slop-only capabilities

```
// sql in the actor: synchronous
sql.all(q, ...params): Row[]                          sql.one(q, ...params): Row
| null
sql.run(q, ...params): { changes, lastRowid }      sql.transaction<T>(fn: ()
=> T): T   // a SAVEPOINT inside the turn
// sql in pages: structured, over exposed names only (section 5.1)
slop.sql.select(name, { columns?, where?, orderBy?, limit?, offset? }):
Promise<Row[]>
slop.sql.watch(name, query, onRows): Unsubscribe
// integers cross every boundary as numbers when |n| <= 2^53 - 1, otherwise
as decimal strings; never BigInt

// files
files.read(path, { at?: n | ref }): Uint8Array      files.text(path, opts?):
```

```
string
files.write(path, bytes | string, { type? })       files.remove(path)
files.list(prefix?, { zone?: 'tree' | 'assets', at? }): FileInfo[]
files.snapshot({ message, ref? = 'main', image? }): { n }     // the image is
taken after the turn

// log
log.since({ epoch, rev }, { select? }): { commits, reset }
log.watch({ from?: { epoch, rev }, select? }, onCommit): Unsubscribe       //
pages, doors

// actions (pages)
slop.actions.<name>(args): Promise<json>           // errors carry { code,
message }

// stores
stores.open(id): StoreHandle    // .call(action, args), .docs, .sql (reads,
by grant), .log, .files
stores.create({ id?, type?, app?, from?: { store, n?, image? } }):
StoreHandle    // platform 'stores:create'

// spawn (section 9.7). In v3.0 a delegate cannot spawn further delegates.
spawn.view({ app, store, scope?, caps, level }): { url, token }    // pages:
an iframe for a child app
spawn.handle({ store, scope?, caps, level, ttl }): DelegateToken      //
actors: pass narrowed rights on

// alarm
alarm.at(timeMs, name, data?)      alarm.every(name, cron, data?)
alarm.cancel(name)      alarm.list()

// fetch: the WHATWG shape, restricted to slop.fetch.allow origins, 10 s, 5
MiB responses
await fetch(url, init) -> { status, headers, text(), json(), bytes() }

// secrets: per app
secrets.get(name): string          // only names listed in slop.secrets and
approved

// platform (each method needs its permission)
platform.mounts.request({ host, path, store })   // 'mounts:request': an
admin approves
platform.mounts.set({ host, path, store })       // 'mounts:write'
```

```
platform.grants.set(store, subject, level)      // 'grants:write', on stores
it owns
platform.refs.move(store, ref, n)               // 'refs:write'
platform.ask({ text, options? }): Promise<answer> // 'ask'; ends the turn
```

7.3 Rules every capability follows

- **Absence is normal.** The `use()` result is `null` when the manifest does not declare a capability, when the version is not approved, or when the principal's kind may not have it. Pages must still render.
- **Errors share one set of codes:**
  - `permission_denied`, `invalid_argument`, `not_found`, `conflict`;
  - `quota_exceeded`, `rate_limited`, `unavailable`;
  - `query_too_slow`, `actor_error`, `consent_required`.

  The contract's own codes keep their meanings.
- **Every call is checked on the server.** A page's `use()` result is only a mirror; the kernel checks `can()` (section 9.5) on every RPC, query and write.
- **Limits are the contract's** for contract capabilities, and section 4.8's for the rest.

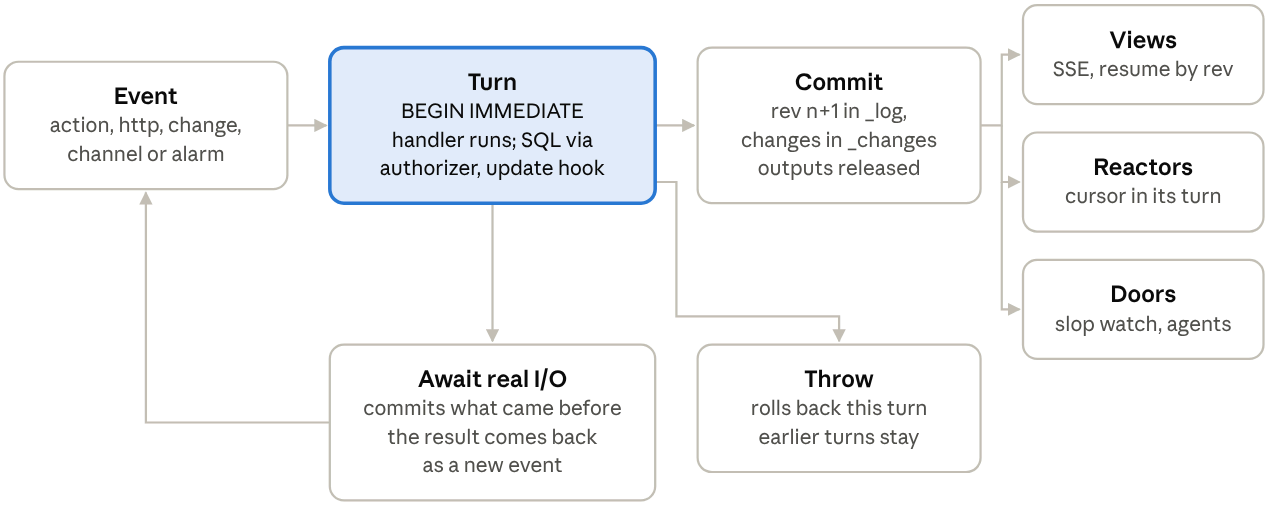
8. Events

The event system is each store's log, read with a cursor. A commit and the events describing it land in the same transaction, so there is nothing to keep in sync and nothing to lose between them.

- **Views** read the log as a stream.
- **Reactors** read it with a cursor saved alongside their own writes, which gives exactly-once effects on stores.
- **Agents** read it from the CLI.
- **Other stores** read it by grant.

Two things are not in the log: channel messages, which are live by design, and alarms, which are wake-ups rather than changes.

One event, one turn, one commit, then views, reactors and doors hear of it



a turn's path · event, transaction, commit, fan-out

An `await` on real I/O ends the turn and commits. A throw rolls back only the current turn. Nothing leaves a turn before its commit succeeds.

8.1 The event

A subscriber receives commits, each with its changes filtered to what it asked for and may read:

```
{ "store": "ws-team", "rev": 4182, "at": 1793461200000,
  "principal": "usr_02657c47", "via": "page", "app": "blocks@17", "step":
"stp_9k2",
  "changes": [
    { "kind": "doc",  "key": "chunks/c41", "op": "update", "version": 88,
"patch": { "$splice": [12, 0, "…"] } },
    { "kind": "row",  "key": "tasks:311",  "op": "insert" },
    { "kind": "table","key": "tags",       "op": "write" } ] }
```

- **Document changes** carry the new `version`. A view that mirrors the path also gets the body, or a diff against the version it holds (SLOP-009's diff sync), computed when the event is sent.
- **Row and table changes** carry no values; readers re-read.
- **One selector grammar** serves `on` in `actor.js`, `log.watch` and `slop watch`:
  - `doc:<glob>`, `file:<glob>`;

  - `table:<name>`, which matches both `table` and `row` changes for that table;
  - `ref:<name>`, `snapshot`, `schema`;
  - `msg:<topic>`, for durable topics in v3.1;
  - `*`.

  A `store:<id>` prefix reads another store, by grant. `channel:<topic>` is accepted only in `on` and receives live channel messages, which are not in the log.
- **What a handler receives:** the commit, with `changes` filtered to what its selector matched.

8.2 Subscribers

Subscriber	How it subscribes	Cursor kept in	Delivery
View (a page)	its `db` subscriptions, `sql.watch` read sets, `slop.log.watch`	the page, sent back as `after=<epoch>.<rev>` on reconnect	at least once, deduplicated by rev; past retention or after a restore, `reset`
Reactor (`on` in `actor.js`)	selectors in `on`	`_cursors` in the reacting store, committed in the reactor's own turn	exactly once for writes to that store (reactors are synchronous); never its own commits
Door (`slop watch`, `slop comments wait`)	flags	the caller (`--from`)	at least once
Another store's reactor (`'store:<id> <selector>'`)	the same `on` syntax	`_cursors` in the reacting store, keyed by the source store	as for reactors; needs a `view` grant for `app:<id>` on the source

A reactor is a turn like any other, so its writes, its cursor move and its own events commit together. If the actor crashes mid-turn, the cursor did not move and the event is delivered again. This is Ultimate's durable-events rule.

8.3 Channels

Each store has one channel: the contract's `room`.

- **Presence:** each view and agent peer sets a JSON object of at most 4 KiB. Peers see joins, leaves and updates.
- **Messages:** `emit(topic, data)`, at most 4 KiB. A topic is admin-only unless the manifest opens it: `"topics": { "cursor": "interact" }`. The long form `{ "level": "interact", "durable": true }` is reserved for v3.1.
- **Live only:** messages are not stored, and a peer that was away does not get them.
- **Durable topics (v3.1):** each message is stored as a `msg` change, so chat-like streams survive and can trigger reactors.
- **Actors in the channel:**
  - `on: { 'channel:<topic>' }` receives messages;
  - `use('room').emit` sends them after the turn commits;
  - agents join with `slop room watch --presence '{...}'`.

8.4 Alarms and schedules

- **Where alarms live:** `every` in `actor.js` declares recurring alarms, and `use('alarm').at(t, name, data)` sets one-off ones. Both live in `_alarms` in the store.
- **Waking only what is due:** each store's next alarm time is mirrored to `schedules(store, next_at)` in `system.db`, so the scheduler wakes only the stores that are due, without opening every store file.
  - The mirror is written before the store's commit. A wake that finds nothing due is harmless and refreshes the mirror.
  - On start, the kernel reconciles `schedules` with the stores that changed since the last clean stop.
- **How an alarm runs:** as one or more turns on the woken actor. It is marked `running` until its handler finishes, then removed. Delivery is at least once (section 6.3).
- **Missed alarms:** if the server was down, an alarm runs once on start with `ctx.event.late = true`, not once per missed period.
- **In the workbench,** alarms run off a virtual clock that can be paused and advanced (section 14).

8.5 Calls between stores

- `use('stores').open('journal').call('publish', args)` sends an action to another store's actor. It needs an `interact` grant for `app:<this app>` on that store (or whatever level the target action requires).
- The call is an `await`, so it ends the caller's turn (section 6.3). The callee runs it as its own turn, with `ctx.principal` = `app:<caller app>` and `ctx.event.onBehalfOf` = the original principal.
- There are no distributed transactions. A workflow that must not half-happen writes an intent document first and lets a reactor finish it. That is the outbox pattern, and the log makes it cheap.

8.6 The system store

Platform changes are commits in the system store's log. That log lives in `system.db` beside the registry tables, so each of these changes and its log entry commit together:

- store created, forked or deleted;
- mount set or undone;
- grant set or revoked;
- version approved;
- token issued;
- sign-in;
- activation passed or failed.

The Activity surface merges the system log with the logs of the stores in view. A project's activity is the same merge, filtered to the project's stores.

8.7 Guarantees in one place

Property	Guarantee
Order within a store	total, by rev, within an epoch
Order across stores	none; each commit names its store, epoch and rev
A commit's visibility	all of its changes at once, never part

Property	Guarantee
Views	at least once, resumable until retention or a restore
Reactors on their own store	exactly once; synchronous handlers only
Async action continuations	lost if the actor crashes; the caller gets `actor_error: lost`
Effects outside a store (fetch, sample, other stores)	at least once: make them idempotent with a request id
Channel messages	at most once (durable topics in v3.1)
Alarms	at least once; late alarms run once

9. Identity and access

Every check asks one question: may this principal do this operation on this store, with this app's code? The answer combines two things:

- the principal's **level** on the store, compared with the **rule** for the path, table, action or topic;
- whether the **app version** is allowed the capability involved.

Guests, share links, agent tokens and child apps are all principals or grant subjects. They need no mechanism of their own.

9.1 Principals

Kind	Id	Comes from	Highest level	Notes
user	`usr_…`	OIDC sign-in	any	platform role: `admin`, `member`, `pending` or `disabled`
guest	`vis_…`	a cookie on a site host, made on the first visit to a public mount	`interact`	30 new guests per IP per hour; deleted after 90 days unseen; can be claimed by signing in (asked, never automatic)

Kind	Id	Comes from	Highest level	Notes
agent token	`tok_…`	device-code login (`slop auth login`), acting for a user	the user's level, capped by the token's `max_level` and store list	acts as its user: `slop_principal()` and `{self}` resolve to the user, and the log's `via` names the token
link	`lnk_…`	a share link's bearer token	`interact`	opening it gives the opener a grant whose `via` is the link; revoking the link removes those grants
app	`app:<id>`	an actor acting in its own name	`owner` on stores running it; elsewhere what grants to `app:<id>` give	calls between stores, platform operations
delegate	`dlg_…`	`spawn` (section 9.7)	the parent's level or lower, on one store, under one path prefix	acts for the user who opened the parent; expires; cannot create delegates in v3.0
anonymous	`anon`	a request to a `public` HTTP route without a session	none	reaches only routes marked `public` ; the actor decides the rest
system	`system`	the kernel	all	migrations, activation, bookkeeping

9.2 Sessions

v2's working identity code is kept and rewritten without compatibility:

- **Main host:** OIDC sign-in with PKCE, server-side sessions with CSRF tokens, and absolute expiry.
- **CLI:** device-code login into `~/.config/slop/` .
- **Site hosts:** their own session cookies, handed over from the main host with a one-time code bound to the browser that started it (SLOP-011).

- **New users** start `pending`. An admin lets them in as `member` or `admin`.

9.3 Levels and grants

- **Levels** are the contract's: `view` < `interact` < `admin` < `owner`.
  - A store's creator is its `owner`.
  - Platform admins are `admin` on every store.
  - Everyone else has the highest level any of their grants gives.
- **Grant subjects:** a principal id (`usr_…`, `vis_…`, `app:<id>`), `*signed-in` (every member and admin) or `*anyone` (anyone, guests included).
  - A subject can hold several grants on one store, from different sources (`via`: direct, a link id).
  - Its level is the highest of them.
  - Revoking a link removes only the grants with that link as `via`.
- **`instantiate` grants on an app** say who may create new stores that run it. They replace v2's `create_policy`.
  - `per_day` caps creations per subject; the default is 50.
  - Guests are capped at 3 a day in total, across all apps.
- **Grants change live.** Open views whose level changed are closed and reopened at the new level (v2 behaviour). Actors see the new level on the next event.

9.4 Rules

Rules map an operation to the minimum level it needs. They come from the app manifest; admins cannot weaken them per store, but can raise them.

Operation	Rule source	Default
Read or write a document	`capabilities.db.rules`, path patterns with `{self}`	read `view`, write `interact`; `data/users/{self}/…` private to its owner
SQL on a table or view	`slop.sql.expose`	not exposed
Call an action	`ctx.need(level)` inside it; `slop.api` documents it	none: the action decides
HTTP route	`level` on the route	`view`

Operation	Rule source	Default
Channel topic	`capabilities.room.topics`	`admin`
Comment	contract	`interact`
Files: read the tree	none	`view`; the assets zone follows `db` rules for `assets/`
Files, refs, snapshots: write	platform	`admin`

9.5 The check

```
can(p, store, op, target):
  if p.kind == system: allow
  who  := p.user if p.kind in (token, delegate) else p   # tokens and
delegates act for a user
  lvl  := level(who, store)                 # highest grant, role, ownership
  lvl  := min(lvl, cap_for_kind(p.kind))    # guest and link at most
interact; token at most its max_level
  if p.kind == delegate: require store == p.store and target under p.scope;
lvl := min(lvl, p.level)
  app  := the store's app at its running version
  need := rule(app.manifest, op, target, who) # {self} resolves to who.id
  if lvl < need: deny permission_denied
  cap  := capability_of(op)                 # db, sql, sample, fetch,
platform:...
  if cap is not always-on and cap not in app.manifest: deny (use() returned
null)
  if needs_approval(cap) and not approved(app, cap): deny
  if p.kind == delegate and cap not in p.caps: deny
  if needs_consent(cap) and consent(who, app, cap) != granted: deny
consent_required
  allow
```

- `level()` reads `system.db` once per view or turn, then follows grant changes from the system log.
- The same function serves pages, actors, SQL connection functions (`slop_can_read`), doors and mounts. There is no second check anywhere.

9.6 New stores from a mount: create and fork-on-open

A mount can name its store in the path (`/blocks/:store`). When the path names no existing store:

- **`create`** : a principal with an `instantiate` grant on the mount's app sees "Create a new one". The new store gets a random id, the app's type, and the creator as owner.
- **Fork-on-open (v3.1):** every visitor, guests included, gets their own fork of a template store on first visit, remembered in `session_forks` . That is the essay's "per-visitor desktops" by link. It is designed now so that mounts and grants need no change later.

9.7 App capabilities, approval, consent and narrowing

- **Approval:** `sample` , `mcp` , `fetch` , `secrets` , `spawn` and `platform` need an admin's approval of the app version that declares them (`approvals` in `system.db`). Until then `use()` returns `null` for them, and the app still runs.
- **Consent:** `sample` and `mcp` also ask each user once per app, on first call, because they spend that user's or the host's credentials.
- **Narrowing (spawn):** a parent app hands a child a delegate token with:
  - one store;
  - a path prefix (`scope`);
  - a subset of its capabilities;
  - a level no higher than its own.

  Examples:
  - Claude Computer opens a calculator made inside it with `db` and `room` only, on `apps/calc/` of its own store.
  - Pinup publishes with `platform: mounts:request` , not `mounts:write` .

  In v3.0 a delegate cannot create further delegates. Revoking the parent's grant revokes its delegates.

9.8 Defaults by principal kind

What v2 hard-coded becomes a default per principal kind, fixed in v3.0:

Capability	guest	member	admin	agent token
`db` , `sql` (exposed), `room` , `user` , `actions` , `downloads`	yes	yes	yes	yes

Capability	guest	member	admin	agent token
`assets` upload	no	`interact` and up	yes	yes
`comments`	no	yes	yes	yes
`sample`, `mcp`	no	if approved and consented	if approved	no
`platform`	no	own stores, if approved	yes	if the token allows

10. Serving

A mount maps a host and path to an app on a store. That is all it does; everything clever happens in the app. A request is resolved in four steps:

1. Pick the mount.
2. Pick the store (fixed, from the path, created or forked).
3. Pick the app version (the mount's ref).
4. Answer from the actor's `http`, the snapshot's files, or a view shell, in that order.

Pages always run on their store's own origin, so browser storage and cookies never cross stores.

10.1 Hosts and origins

Host	Example	Serves
Main	`slop.computer`	the shell, sign-in, the doors' HTTP API, safe mode `/_/`
Site hosts	`site.slop.computer`, `lab.example.org`	mounts; a small shell page for live apps; sessions for signed-in people and guests
Sandbox	`s<label>.slop.run`	one origin per store: `label` = 20 hex digits of HMAC(server key, store id); the app version in the path `/v/<n>/`

- The sandbox domain is a separate registrable domain, so cookies set on the main host never reach app code.

- A store's origin never changes, so `localStorage` and IndexedDB survive new versions (the contract's promise).
- Sandbox responses send `frame-ancestors` for the main and site hosts only.

10.2 Mounts

```
CREATE TABLE mounts (
  host TEXT NOT NULL, path TEXT NOT NULL,          -- '/blocks/:store',
'/journal/*'
  store TEXT,                                      -- a fixed store, or NULL
with store_param
  store_param TEXT,                                -- the path parameter
naming the store
  type TEXT,                                       -- with store_param: only
stores of this type match
  on_missing TEXT NOT NULL DEFAULT 'none' CHECK (on_missing IN
('none','create')),
  create_app TEXT,                                 -- with 'create': the app
new stores run
  mode TEXT NOT NULL DEFAULT 'live' CHECK (mode IN ('live','static')),
  changed_by TEXT, changed_at INTEGER NOT NULL,
  PRIMARY KEY (host, path));
```

- **A mount names a store, never an app or a version.** The code is the store's app at its running version, so a store always runs exactly one version (D4).
- **`live`** runs the app as a view: a shell page on the site host, the app on the store's origin.
- **`static`** serves the app's files directly on the site host, cacheable and without a session. It suits blogs, docs and landing pages, and can still call the actor's `http` routes.
- **Every change commits in `system.db`** together with its log entry, so `slop mount undo` and the Activity surface work as v2's route log did.
- **What goes away:** `record`, `page_from` and data bindings are app code now. A small "site" app with a router, reading a content store through `use('stores')`, replaces all three.

10.3 Resolving a request

```
resolve(host, path):
  m := longest matching mount on host (literal segments beat :params beat *)
  store := m.store or params[m.store_param]
  if store is missing, or m.type is set and the store's type differs:
      none   -> the site's 404 page
      create -> the shell offers "Create" if can(p, m.create_app,
instantiate); else 404
  app, v := the store's app and running version (activating a pending push
first)
  rest := path after the mount prefix
  1. rest matches an http route in app@v's actor.js   -> an actor turn, its
response
  2. rest names a file in app@v's snapshot tree        -> the file (static)
or the file on the store's origin (live)
  3. mode == live                                       -> the shell page,
which opens a view on the store
  4. the app has a page and rest has no extension      -> index.html (client-
side routing)
  5. otherwise                                          -> 404
```

10.4 Caching

SLOP-012's rules carry over: what a URL names decides how long it may be kept.

URL names	Example	Cache
A version or a hash	`/v/12/app.js`, `/_blob/<sha>`, `/_r/journal/15.js`	`immutable`, one year
A ref or a place that moves	`/journal/`, `style.css` beside a page that follows `main`	60 s, then revalidated in the background
An actor response	`/api/tasks`	whatever the handler sets; default `no-store`

Images take `?w=` for widths 160 to 1920 (SLOP-012). Text responses are gzipped.

10.5 Views

The page transport is SLOP-009's, unchanged in shape.

1. The shell (main host or site host) asks for a view: `POST /api/views {store}`, with its session. A preview of another app version asks for a fork made for that purpose, never the live store.
2. The kernel answers with:
   - the view id;
   - a secret;
   - the store's origin URL with `/v/<n>/`;
   - the capabilities this view may use;
   - the principal's level.
3. The shell loads the iframe and sends the secret by `postMessage`, checked against the frame's origin. The secret is never in a URL.
4. The page's runtime (`/_slop/claude.js`, injected into `index.html`) uses one event stream and one RPC endpoint:
   - `GET /api/views/<v>/events?after=<epoch>.<rev>`, read with fetch streaming, because `EventSource` cannot send an `Authorization` header;
   - `POST /api/views/<v>/rpc {method, args}`.

   Every call carries `Authorization: Bearer <secret>` and no cookies.
5. The stream carries:
   - `hello`;
   - `commit` (filtered to the view's subscriptions and rules);
   - `rows` (watch results);
   - `peers` and `room`;
   - `sample` deltas;
   - `reload` (a new version activated);
   - `revoked`;
   - `reset`.
6. The view lives while its stream is connected. The secret is renewed every 45 minutes through the shell.

Agents do not need views to read or write. They can open one (`kind: agent`) to join a channel with presence.

11. The shell and surfaces

The shell's top level shows only what people own and decide about: **Data, Apps, Sites, People and Activity**. Mechanisms are tabs on an object; plumbing is hidden. The shell is built like Ultimate's:

- every screen is a surface made from the same parts apps use;
- apps add surfaces, panels and openers through slots;
- an agent is docked on every surface.

11.1 What the UI makes explicit

Where	What
Top-level surfaces	Data, Apps, Sites, People, Activity, plus surfaces apps add
Tabs on a store	Contents (documents, files, tables), SQL, Log, Snapshots and refs, Members and links, Mounts, Channel (who is here), Actor (state, alarms, reactors, errors), Limits
Tabs on an app	the store tabs, plus Manifest, Approvals, Stores running it, API
Hidden, in a debug drawer	origins and labels, view ids and secrets, leases, consents, sign-in handoffs, site sessions, blob hashes, raw revs
No longer nouns	page vs data, target, instance, tool, project as a table: a project is a store of type `project` with a brief and a list of stores

11.2 The built-in surfaces

- **Data:** every store that is not an app, grouped by type.
  - Browse documents as a tree and as a table of a collection's fields.
  - Browse files and app tables, run SQL with the store's rules or as admin, and edit a document or row.
  - See history per document, snapshot, fork, import and export.

    - Use "Open with" for apps whose `opens` matches the type, and see who is in the channel now.

  This is the database management v2 lacked.
- **Apps:** every store with `app.json`.
  - Launch, see versions with their messages and authors, and diff two versions.
  - Read the manifest with its capability requests and approve them.
  - See the stores that run the app and where it is mounted.
  - Read the API file agents use.
- **Sites:** mounts by host, as a list and as a site map with links drawn between pages. A broken link is shown orange.
  - Moving a mount is one click, and undo is the next.
  - Safe mode `/_/` stays a server-rendered page outside the shell, for when the shell itself is broken.
- **People:** users, guests, agent tokens and share links.
  - A matrix of who reaches which store at which level.
  - Pending approvals of app versions, consent records, and the queue of questions agents asked (section 12.5).
- **Activity:** the merged log.
  - Commits by store, principal, app and step, with keep and revert for steps.
  - Comments waiting for an answer.
  - Actors that are awake, broken or behind on a reactor; alarms due.
  - Activation results.

11.3 Slots

Slot	Filled by	Shows	Ships in
`surface`	an app's `slop.surfaces` entry with `slot: surface`	a full-screen app in the switcher	v3.0
`opener: <type>`	apps whose `opens` is that type	the store's contents through that app, inside Data	v3.0

Slot	Filled by	Shows	Ships in
`panel:<surface>`	apps	a side panel on a built-in surface (for example, a Blocks outline beside Data)	v3.1
`inspector:<type>`	apps	a better view of one document or row of that type	v3.1
`event:<kind>`	apps	how a change of that kind reads in Activity	v3.1
`command`	apps	an entry in the command palette, running an action	v3.1

Each slot item renders inside its own error boundary. A broken one shows an error card, and the rest of the surface keeps working.

11.4 Two kinds of surface

- **Iframe apps:** a full page of the app on the store's origin, through a view. They can do anything and cost the most. Claude Computer, Blocks and Pinup are this kind.
- **Declarative surfaces:** an actor or a sandboxed page script returns a **widget tree**, and the shell draws it with its own components. They are safe to put inside the shell, load fast, and are easy for agents to write. This is the idea behind go-go-os's `ui.card.v1`, go-go-host's `ui.dsl`, Blocks' Fruit views and Ultimate's surface parts.

Widget pack `slop.ui.v1`, deliberately small:

- Layout: `stack`, `row`, `grid`, `section`.
- Text and display: `text`, `heading`, `markdown`, `code`, `json`, `badge`, `link`, `image`, `chart` (line, bar, scatter from rows).
- Data: `table` (rows with sort and select), `list`.
- Input: `form` with `field`, `select`, `toggle`, `button`.
- Embedding: `frame` (an iframe view of another app).

```
{ "t": "stack", "c": [
  { "t": "heading", "text": "Open tasks" },
  { "t": "table", "rows": [{ "id": 1, "title": "Write brief" }], "select":
"selected" },
  { "t": "button", "label": "Done", "on": { "action": "markDone", "args": {
"from": "selected" } } } ] }
```

- **Events** go back as `{action, args}`. The shell calls the actor's action, or sets local state keys named in the tree (`select`, `value`). It then asks for the tree again.
- **Every tree is validated** against the pack's schema before drawing. An invalid tree shows an error card with the path of the bad node.
- **The pack grows by version,** not by special cases. When a surface needs more than the pack offers, it becomes an iframe app; go-go-os showed that a DSL chasing every widget falls behind.

11.5 The agent dock

- Every surface has a dock for the agent (Ultimate's `Dock`; the Form Skate Agent panel's look), collapsed to a rail when unused.
- **The agent sees what is selected:** the store, document, row, log entry or mount in focus is part of its context.
- **It works through one `eval` tool:** JavaScript run against the doors' API as a scope, in Auto, Ask or Read-only mode (section 12.3).
- **Every step is reviewable:** each agent step is a step in the log, so Activity can keep or revert it.

11.6 The shell is an app

The built-in surfaces ship as an app store, `shell`, with the platform capability and an approval made at install. Changing the shell is a push to that store, so it can be forked, reviewed and reverted like anything else. If the shell's new version breaks, the fallback rule (section 6.4) brings back the last working one, and safe mode is always there.

12. Agents and doors

Agents use the same verbs people do, and from one definition the kernel generates:

- the CLI;
- the HTTP API;

- the MCP tools;
- the docked agent's `eval` scope.

This is Librarian's "three doors into one implementation", with a fourth door for JavaScript. An agent's work is grouped into steps that a person can keep or revert. When it needs a decision, it asks a question that waits in the UI.

12.1 One verb table

```
verb('docs.set', {
  args: { store: 'string', path: 'string', data: 'object', ifVersion: 'int?'
},
  needs: { capability: 'db', op: 'write', target: a => a.path },
  run: (k, a, ctx) => k.store(a.store).turn(ctx, t => t.docs.set(a.path,
a.data, { ifVersion: a.ifVersion })),
  doc: 'Write a whole document. Fails with conflict if ifVersion is given and
differs.',
})
```

- **CLI:** the verb `group.name` becomes `slop group name <positionals> --flags`, built with Glazed in Go. Output can be a table, JSON or YAML.
  - Short aliases cover the loop agents run most: `slop push` and `slop pull` (`files push`, `files pull`), `slop call` (`actor call`), `slop watch` (`log watch`), `slop export` and `slop import` (`store export`, `store import`).
  - `slop auth` and `slop lint` are local commands, not verbs.
- **HTTP:** `POST /api/v1/<group>.<name>`, with the args as JSON.
- **MCP:** a tool `<group>_<name>`, with its JSON Schema made from `args`. MCP tool names allow only letters, digits, `_` and `-`.
- **Eval scope:** `slop.<group>.<name>({...})`.

The same `needs` runs `can()` before the handler, whichever door the call came through.

The verbs, grouped:

Group	Verbs
Stores	`store.ls`, `store.new`, `store.fork`, `store.rm`, `store.export`, `store.import`, `store.meta`

Group	Verbs
Content	`docs.get`, `docs.list`, `docs.query`, `docs.set`, `docs.update`, `docs.replace` (string replace in a field), `docs.rm`, `docs.batch`
Files	`files.ls`, `files.cat`, `files.pull` (a store's tree into a folder), `files.push` (a folder into the tree, then a snapshot), `files.rm`
SQL (admin)	`sql.query`, `sql.exec`, `sql.schema`
History	`log.show`, `log.watch`, `snap.ls`, `snap.new`, `ref.ls`, `ref.move`, `step.start`, `step.end`, `step.ls`, `step.keep`, `step.revert`
Actors	`actor.call`, `actor.status`, `actor.logs`, `actor.restart`, `alarm.ls`, `alarm.fire`
Sites	`mount.ls`, `mount.set`, `mount.rm`, `mount.undo`
People and access	`whoami`, `grant.ls`, `grant.set`, `grant.rm`, `link.new`, `link.rm`, `approve.ls`, `approve.set`, `token.ls`, `token.rm`, `secret.ls`, `secret.set`
Talking	`comments.ls`, `comments.wait`, `comments.reply`, `room.watch`, `room.emit`, `ask`, `answer`

12.2 Outside agents: Claude Code, Pi, Codex

The v2 loop stays, with fewer nouns:

```
slop auth login --server https://slop.computer --name "claude on my laptop"
slop step start -m "pricing page, three tiers"            # later commits
from this folder join the step
slop pull pricing ./pricing && $EDITOR ./pricing            # the store's
tree into a folder
slop push ./pricing -m "tiers: gap 48 -> 24px"            # tree write +
snapshot; the folder remembers its store
slop push ./new-page --store new-page --create -m "first"  # a new store; it
has app.json, so it is an app
slop mount set site.slop.computer /pricing pricing         # a mount names a
store
slop comments wait --store pricing                          # blocks until
someone comments; prints anchor and element
slop sql query journal "SELECT slug, title FROM posts ORDER BY at DESC LIMIT
5"   # admin only
```

The MCP server lists about twenty tools: the verbs agents use most, plus `verb`, which runs any verb by name. Tool lists stay short, and nothing is out of reach. App actions become tools in v3.1 (section 13.1).

12.3 The docked agent and its one tool

The agent in the dock follows the Form Skate Agent's design:

- **One tool, `eval`.** Its input is the body of an async JavaScript function. It runs in a sandbox (QuickJS in the workbench, goja on the server) with `slop` (the verbs), `selection` (what is in focus) and `console` in scope, and returns YAML.
- **Three modes:**
  - **Auto** runs each step.
  - **Ask** shows the code and waits for **Run this step** or **Decline**.
  - **Read-only** gives a scope whose writing verbs throw.
- **Every `eval` is a step.** Its commits are tagged with the step id, so Activity shows the step and "revert" undoes it (section 12.4).
- **The model** comes from the host's model backend: Pi credentials through geppetto in Go (SLOP-010), or an API key or a fake backend in the workbench.

12.4 Steps: keep and revert

A step groups commits made for one purpose, across stores. It is started by `slop step start`, by one `eval`, or by a push.

Reverting a step applies the inverse of each change, newest first, as a new step:

Change	Inverse
`doc`	write `old`, which is kept for commits inside a step
`file`	restore the old hash
`ref`	move it back
`snapshot`	none: it stays in history
store created	delete the store, after a confirmation
mount or grant	restore the previous value, from the system log
`row`, `table`	not revertible in v3.0: the revert stops and names the tables. Revert through the preupdate hook is v3.1
`schema`	not revertible: migrations are forward-only

Keep marks the step reviewed. Unreviewed agent steps are badged in Activity.

12.5 Questions

`slop ask "Publish /pricing now?" --options yes,no --store pricing` creates a question, a document in the `system` store, and blocks until it is answered or times out. Pages and actors can ask too, through `platform.ask`.

Questions appear in People and in the dock, in the shape of go-go-os's plz-confirm requests: confirm, select, form or upload. The answer is a commit, so an agent that was away sees it in the log.

12.6 Agent conversations are stores

A conversation with the docked agent is a store of type `conversation`, following Pi Durable's model:

- the transcript is documents;
- tool calls and their results are committed before they show;
- forking at a message is a store fork;
- compaction notes are files.

The agent's runtime is an actor of the `agent` app, so a conversation keeps going when nobody has the page open, and an alarm can wake it ("remind me tomorrow").

13. Hosting the app builders

The first tenants are apps that build apps. Each runs unchanged as a page, because it is written against the claude.ai contract. Each then gains what a browser tab cannot give it:

- work that runs with the page closed;
- real URLs for what it makes;
- connectors on the server;
- outside agents as peers.

The gains come from an `actor.js` and a manifest added next to the upstream files, never from changes inside them.

13.1 What each needs and gets

App	Uses today	Runs on v3 as	What v3 adds
Claude Computer	`db` (`desks` and files), `room`, `sample`, `downloads`. Its apps run in in-page sandboxes and its programs in workers. `jobs.<name> = {every | at, run, ask}` run only while a page is open	app `computer`; one store per desktop, type `computer.desk`	an actor that runs due jobs as alarms, with the job code in a spawned child sandbox; "Pin to a URL" (13.2); desk files as store files with URLs

App	Uses today	Runs on v3 as	What v3 adds
Claude Blocks and Meta Blocks	`db` (one Hypha log in chunk documents of up to 500 ops), `room`, `sample`, `assets`, `mcp` (`host:librarian`, `host:claude_browser`)	app `blocks`; one store per workspace, type `blocks.workspace`	an actor that loads `hypha.js` headless and folds the log in a reactor, so formulas, jobs and holes advance with no tab open; agents take turns with the turn protocol through `slop docs`; Librarian reached as a store, not a desktop-only connector
Pinup	`db` (one document per layer), `room`, `downloads`; publishing compiled pages needed slop's own API	app `pinup`; one store per design, type `pinup.design`	publish creates or updates an app store and requests a mount (`platform:stores:create`, `mounts:request`); live preview is a spawned view of the compiled app on a fork of the content store
Librarian (Hypha)	its own folder with CLI, HTTP and MCP doors, a changes feed, mutable memory and immutable storage zones, HTML lenses	app `librarian`; store type `librarian.folder`	the verbs (`find`, `get`, `put`, `update`, `promote`, `supersede`, `neighbors`, `maps`, `changes`) become actions; FTS5 tables for `find`; the storage zone becomes snapshots; lenses are apps that open the type

App	Uses today	Runs on v3 as	What v3 adds
The shell (Ultimate's model)	n/a	app `shell`	surfaces, slots, the agent dock, approvals at install (section 11.6)

Actions are reachable from every door. An app's actions, documented in its `slop.api` file, can be called from the CLI and over HTTP: `slop call knowledge find --query kickoff`. In v3.1 the MCP door also lists them as tools named `<store>__<action>`. That is how Meta Blocks' lens and outside agents reach Librarian without a desktop-only connector.

13.2 How content becomes an app

In the essay's terms, software is content. The platform therefore makes the step from "some files in a store" to "an app at a URL" short, explicit and reversible. There are three moves, each a capability with its own approval:

1. **Spawn: alive, nested, no URL.** A parent app runs a child app from files in its own store, with a delegate token: one path prefix, a capability subset, a level no higher than its own (section 9.7).
   - In a page this is an iframe on a child origin.
   - In an actor it is a child sandbox.
   - This is how Claude Computer's inner apps and programs run once Computer opts in, and they get real isolation instead of same-page sandboxes.
2. **Publish: a new app store.** The parent copies a subtree of its files into a new store (`stores.create` plus `files.write` plus `files.snapshot`), which now has its own versions, grants and log.
   - The new store records `made_by: <parent store>@<rev>` in `_meta`, so credit flows upstream: the essay's "echo" problem has at least a provenance trail.
3. **Mount: a URL.** The parent requests a mount (`mounts:request`). An admin, or an owner of the site host, approves it in People. With `mounts:write` approved for that app, the request applies at once.

The reverse also works. Any app's tree can be copied into another store as content (`files.pull` and `files.push`, or `stores.create({from})`) to be remixed. The copy keeps `made_by`.

13.3 The three distribution channels

The essay names three ways software travels. The platform serves each:

- **The link:** a mount. In v3.1, fork-on-open gives each visitor their own copy (section 9.6).
- **The file (v3.1):** `slop export --html` writes one HTML file holding three things:
  - the app's page files;
  - a document snapshot;
  - the smallest build of the workbench kernel (section 14).

  Opened from `file://` it gets no service worker and no OPFS. It therefore runs in memory with its files inlined, and saves by downloading a new copy. Opening it on a slop server imports it as a store.
- **The prompt:** every app has a `PROMPT.md` slot in its tree, holding the paragraph that recreates it. The Apps surface shows it beside the versions.

14. Implementation part A: the browser workbench

The workbench is the whole system running in one browser tab, built to be looked into.

- The kernel lives in a Web Worker with SQLite compiled to WebAssembly.
- Actors run in QuickJS.
- App pages run in iframes served by a service worker.
- Every turn, statement, authorizer decision and event can be inspected, replayed or broken on purpose.

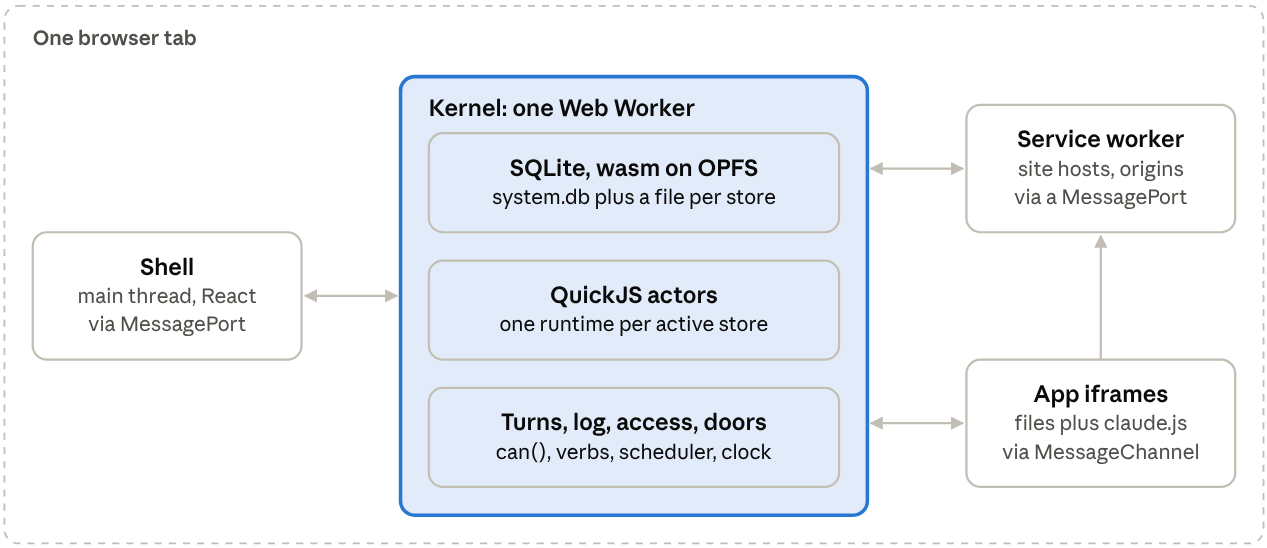
It is also the executable specification. Its conformance suite is what the Go server must pass.

14.1 Threads and parts

Part	Runs in	Does
Shell	main thread, React	surfaces, dock, inspectors. It holds the only port on which the kernel accepts `as` (acting as a chosen principal)

Part	Runs in	Does
Kernel	one dedicated Web Worker	`system.db`, `blobs.db` and every `store.db` through `@sqlite.org/sqlite-wasm` on the `opfs-sahpool` VFS; turns, log, access, views, doors, scheduler
Actors	inside the kernel worker, one QuickJS runtime per active store (`quickjs-emscripten`)	`actor.js` with `use()` bound to synchronous host functions; memory limit, stack limit, and an interrupt handler per turn
Hosts	a service worker	answers `/_h/<host>/<path>` (site hosts) and `/_o/<store>/v/<n>/<path>` (store origins) by asking the kernel over a `MessagePort` the shell transfers to it at start
Pages	iframes	the app's files plus the injected `/_slop/claude.js`, talking to the kernel over a `MessagePort` the shell hands over. The kernel binds each port to one view and its principal
Doors	the kernel worker	the verb table; a CLI console in the shell; the agent's `eval` in a QuickJS sandbox

The workbench runs the whole system in one browser tab



workbench layout · shell, kernel worker, service worker, iframes

The kernel worker owns every database file. The shell, the pages and the service worker reach it only through message channels, the way the Go server will be reached over HTTP.

Fidelity gaps, each marked in the UI where it applies:

- **No WAL:** the wasm build has none, so the workbench uses a rollback journal.
- **One tab:** `opfs-sahpool` holds its files exclusively. The workbench takes a Web Lock at start, and a second tab shows "open in another tab".
- **Simulated origins:** store origins are paths, and iframes get `allow-same-origin` so the service worker can serve their files.
  - A page can therefore reach the shell's window and every page's `localStorage` and IndexedDB.
  - The kernel still binds each port to one view and principal, so a page cannot act as someone else through the kernel.
  - The workbench is still not a sandbox for hostile apps: run only apps you trust there. Isolation is tested on the Go server.
- **One process:** a server restart is a kernel worker restart.
- **Capacity:** sahpool's `addCapacity` is async, so the kernel keeps 16 file slots spare and adds more between turns.

14.2 Ports: the seams the Go server re-implements

The kernel is written against six small interfaces, and the Go server implements the same six. Everything above them is the logic the conformance suite pins down. Opening files is async, because OPFS hands out file handles asynchronously; everything a turn calls is synchronous.

```
interface Storage {                          // sqlite-wasm + opfs-sahpool in
the workbench; mattn/go-sqlite3 in Go
  open(path: string, opts: { create: boolean }): Promise<Db>      // async:
OPFS handles
  remove(path: string): Promise<void>
}
interface Db {                               // every method is synchronous
  prepare(sql: string): Stmt                 // the authorizer runs here
  exec(sql: string): void
  // four arguments: mattn's trampoline drops SQLite's fifth (view or trigger
name); the design does not need it
  setAuthorizer(fn: (action: number, a: string | null, b: string | null, db:
string | null) => 0 | 1 | 2): void
  setUpdateHook(fn: (op: 'insert' | 'update' | 'delete', table: string,
rowid: number | string) => void): void
  setProgressHandler(everyOps: number, fn: () => boolean): void   // Go: a
context deadline instead
  setLimit(id: number, value: number): void
  createFunction(name: string, fn: (...args: SqlValue[]) => SqlValue, opts: {
deterministic: boolean }): void
  vacuumInto(path: string): void             // only outside a transaction, on
a kernel connection
  close(): void
}
interface Stmt {
  bind(params: SqlValue[] | Record<string, SqlValue>): void
  step(): boolean                            // true while a row is available
  row(): Record<string, SqlValue>
  columns(): string[]
  readonly: boolean                          // sqlite3_stmt_readonly
  reset(): void
  finalize(): void
}
type SqlValue = null | number | string | Uint8Array   // integers beyond 2^53
- 1 become decimal strings
```

```
interface Engine { newRuntime(opts: { memoryBytes: number; stackBytes: number
}): JsRuntime }   // QuickJS; goja in Go
interface JsRuntime {
  install(globals: Record<string, HostFn>): void              // use(),
defineActor, require, console
  run(source: string, filename: string): void
  call(fn: JsRef, args: unknown[], budgetMs: number): unknown // throws
Interrupted when over budget
  newPromise(): { promise: JsRef; resolve(v: unknown): void; reject(e:
unknown): void }   // host async ops
  drainJobs(budgetMs: number): void                           // queued
promise jobs, inside the turn
  dispose(): void
}
interface Blobs { put(bytes: Uint8Array): string; get(sha: string):
Uint8Array | null; has(sha: string): boolean }
                                              // workbench: rows in blobs.db;
Go: files on disk
interface Clock { now(): number; at(t: number, fn: () => void): Cancel }   //
virtual in the lab
```

14.3 Module layout

```
workbench/
  kernel/
    ports/          storage-sqlite-wasm.ts, engine-quickjs.ts, blobs-opfs.ts,
clock-virtual.ts
    store/          open, schema, turn, docs, files, changes, snapshots,
refs, fork, export
    sql/            authorizer policy, connection functions, migrations, read
sets, watches
    actor/          supervisor, inbox, turn loop, defineActor, require, use()
bindings, alarms
    log/            commit fan-out, cursors, selectors, reactors
    access/         principals, grants, rules, can(), delegates, approvals,
consent
    serve/          mounts, resolve(), views, sessions, sandbox labels
    doors/          verb table, CLI parser, HTTP-shaped handler, MCP schema,
eval scope
    system/         system.db schema, system store, schedules, steps,
questions
  runtime/          claude.js (the contract in the page), slop extras,
surface renderer for slop.ui.v1
  sw/               service worker: hosts and origins
  shell/            React surfaces: Data, Apps, Sites, People, Activity, dock
  lab/              inspectors (14.4), fault injection, scenario recorder
  conformance/      scenarios (14.6) and the runner
  book/             the textbook (14.5)
```

The stack follows the team's web rules: pnpm, React, TypeScript, RTK Query (with a base query over the kernel port instead of HTTP), and static assets under `/static/`.

14.4 What makes it a workbench

- **Turn inspector:** each turn as a row. Open it to see:
  - the event that started it;
  - every SQL statement with the authorizer's decision per table;
  - the update-hook hints;
  - the commit or rollback;
  - the events released after it.
- **`can()` explainer:** for any denied call, the rule trace:

    - which grant gave which level;
    - which rule needed what;
    - which capability or approval was missing.
- **Principal switcher:** open the same mount side by side as admin, member, guest, agent token and delegate.
- **Virtual clock:** pause, step one turn, jump to the next alarm, or run ten simulated days in a second.
- **Fault injection:**
    - crash an actor mid-turn;
    - fail a commit;
    - slow a statement past its budget;
    - drop a view's stream;
    - expire a cursor past retention.

  Then watch what the guarantees in section 8.7 do.
- **Actor timeline:** per store, its states (cold, waking, active, hibernated, broken), inbox length and the CPU time of each turn.
- **After W8:** a log scrubber (a store's state at any retained rev) and a scenario recorder (a session of door calls recorded as a conformance scenario).

14.5 Look and the textbook

- **Shell:** the Form Skate Workbench system.
    - Kiln style: white ground, one ink, Arial in two weights, 1 px rules, red only for trouble and selection.
    - One window with a dock of tiles, each tile a strip of tabs, layouts saved per person.
    - The surfaces of section 11 are the views placed in the dock.
- **Agent:** the Form Skate Agent system: one `eval` tool, steps shown as collapsible blocks with code, time and result, and Auto, Ask and Read-only modes.
- **Figures:** the Form Skate Stage conventions where the book draws structures (stores, logs, turns) as drafted figures.
- **The textbook,** written with the executable-textbook method in the handbook: each chapter has runnable sketches against the live kernel, computed figures, and checks run mechanically before anyone reads it. Chapters, in order:
    1. a store;
    2. documents;

3. SQL and the authorizer;
4. turns and commits;
5. the log;
6. actors;
7. events and reactors;
8. access;
9. mounts and views;
10. surfaces;
11. agents and steps;
12. apps that make apps.

14.6 The conformance suite

A scenario is a JSON script of door calls, page RPCs, clock moves and faults, with expected results, expected log entries and expected denials:

```
{ "name": "guest cannot read another guest's private doc",
  "given": { "stores": [{ "id": "ws", "app": "notes" }], "grants": [["ws",
"*anyone", "interact"]] },
  "steps": [
    { "as": "vis_a", "call": "docs.set", "args": { "store": "ws", "path":
"data/users/{self}/n1", "data": { "t": 1 } }, "expect": "ok" },
    { "as": "vis_b", "call": "docs.get", "args": { "store": "ws", "path":
"data/users/vis_a/n1" }, "expect": { "error": "not_found" } },
    { "as": "vis_b", "call": "sql.query", "args": { "store": "ws", "q":
"SELECT count(*) n FROM slop_docs" }, "expect": { "rows": [{ "n": 0 }] } } ]
}
```

- **One runner, two backends.** The runner drives the workbench kernel in-process, or the Go server over HTTP. The same files pass on both.
- **SLOP-009's contract conformance pages** join the suite. They run in iframes in the workbench and in a headless browser against Go.
- **Coverage target:** every row of the authorizer table (5.4), every guarantee (8.7), every branch of `can()` (9.5) and every resolve rule (10.3).

14.7 Phases

Phase	Builds	Exit test
W0	kernel worker, ports, Web Lock, `system.db`, store create, open and delete, CLI console	create, list and delete stores from the console; state survives a reload
W1	documents, files, blobs, turns, commits, log, snapshots, refs, images between turns, fork, export and import	the `db` scenarios (the contract conformance checks recast as door calls) pass; every commit shows in the turn inspector
W2	SQL: migrations, authorizer, connection functions, exposure, structured page queries, table changes, live queries	every row of table 5.4 has a passing scenario; pages cannot reach a CTE or raw SQL at all; a watch updates within 100 ms of a commit
W3	QuickJS actors: `defineActor`, turns, actions, `require`, alarms on the virtual clock, synchronous reactors with cursors, activation	crash-mid-turn scenarios show no lost or doubled reactor writes; an async reactor is refused; ten simulated days of alarms run in under 5 s
W4	service-worker hosts, mounts, views, the injected runtime, channels, with one admin principal	the contract conformance pages pass in iframes; Claude Computer, Blocks and Pinup run from their upstream files
W5	principals, grants, links, guests, approvals, consent, one-level delegates, the `can()` explainer	the principal switcher shows one mount correctly at five levels; every denial has a trace
W6	the shell's surfaces, the `surface` and `opener` slots, `slop.ui.v1`	Data browses, queries and edits every store; Blocks contributes an opener
W7	doors as agent tools, the `eval` dock, steps with keep and revert, questions	an agent builds a small app and mounts it; reverting the step restores the documents, files, refs, mounts and grants it touched and deletes the store it created
W8	the textbook's first chapters, the full conformance suite	the book's sketches run; the suite is green and ready to aim at Go

15. Implementation part B: the Go foundation

The Go server re-implements the workbench's six ports with production parts and runs the same kernel logic:

- `mattn/go-sqlite3` for storage;
- goja through go-go-goja for actors;
- the filesystem for blobs;
- `net/http` for hosts.

It is done when the conformance suite passes against it over HTTP. Server-side sandboxing comes in three levels, the first in-process and the second in separate actor-host processes.

15.1 Package layout

```
cmd/slop/                    one binary: `slop serve`, the CLI verbs, `slop
actor-host` (level 2)
pkg/ports/sqlite/            mattn/go-sqlite3: connections, authorizer,
update hook, functions, limits, backup
pkg/ports/engine/            go-go-goja: RuntimeFactory, runtimeowner,
require from a snapshot tree
pkg/ports/blobs/             content-addressed files on disk
pkg/kernel/store/            schema, turns, docs, files, changes, snapshots,
refs, fork, export
pkg/kernel/sql/              the authorizer policy, slop_* functions,
migrations, read sets, watches
pkg/kernel/actor/            supervisor, inbox, turn loop, defineActor,
use() modules, alarms
pkg/kernel/log/              fan-out broker, cursors, selectors, reactors
pkg/kernel/access/           principals, grants, rules, can(), delegates,
approvals, consent
pkg/kernel/serve/            hosts, mounts, resolve, views (SSE + RPC),
static and image serving
pkg/kernel/doors/            verb table -> Glazed commands, HTTP handlers,
MCP tools
pkg/kernel/system/           system.db, the system store, schedules, steps,
questions
pkg/identity/                OIDC, sessions, device login, site handoff
(rewritten from v2)
pkg/models/                  sample() on geppetto with Pi credentials (SLOP-
010)
web/                         runtime/claude.js, the shell app, embedded with
go:embed
conformance/                 the scenario files, shared with the workbench
```

15.2 SQLite in Go

- **Driver:** `mattn/go-sqlite3`, which go-go-goja already depends on. On `SQLiteConn` it has:
  - `RegisterAuthorizer` and `RegisterUpdateHook`;
  - `RegisterCommitHook` and `RegisterRollbackHook`;
  - `RegisterFunc`;
  - `Backup` and `Deserialize`;
  - limits.

It has three gaps, none blocking:

- The authorizer callback receives four arguments and drops the view or trigger name. The design no longer needs it (section 5).
- `RegisterFunc` has only a `pure` flag. That is enough: the `slop_*` functions are registered as not pure.
- There is no `sqlite3_db_config`, so `SQLITE_DBCONFIG_DEFENSIVE` cannot be set. The authorizer denies `PRAGMA writable_schema` and every write to `sqlite_*` instead.

FTS5 needs the `sqlite_fts5` build tag, and the preupdate hook (v3.1) needs `sqlite_preupdate_hook`. `ncruces/go-sqlite3`, which needs no cgo, is the fallback if a gap ever blocks.

- **Connections per open store:**
  - one **writer**, owned by the store's turn loop;
  - up to four **readers**, for page queries, door reads and watches;
  - a **kernel connection** whose `ATTACHED` limit is 1, used only for images between turns.

  Settings: `journal_mode=WAL`, `synchronous=NORMAL`, `busy_timeout=5000`, `foreign_keys=ON`, and `max_page_count` from the store's limit. The authorizer and `slop_*` functions are installed per connection, with the caller set before each statement.
- **Time limits:** statements from pages and doors run with a context deadline of 250 ms; the driver interrupts SQLite when it expires. Actor statements share the turn's budget.
- **Open stores:** an LRU of open store files (default 512) closes idle stores. A hibernated actor's store can stay open for readers.

15.3 Actors on goja

- **One runtime per active store,** built from one `engine.RuntimeFactory` per app version:
  - The capability modules of section 7 sit behind one native `use(name)`, which filters them by the manifest and approvals.
  - `require` resolves files from the app snapshot's tree.
  - `defineActor` records the handlers.
- **The turn loop** is one goroutine per active actor, which owns the inbox and runs JS through `runtimeowner`. For each event it:
  1. begins `IMMEDIATE` on the writer and checks the lease epoch;

2. calls the handler through the owner;
3. runs pending promise jobs;
4. commits;
5. hands the commit to the log broker;
6. releases outputs.

An awaited host operation (fetch, sample, another store's call) runs in its own goroutine. It settles through `runtimebridge`, which posts a continuation event to the same inbox: a new turn.

- **Numbers:** goja numbers are float64, so SQLite integers beyond 2^53 - 1 cross into JS as decimal strings, exactly as in the workbench.
- **Budgets:**
  - A timer calls `vm.Interrupt` when a turn passes 250 ms. The turn rolls back with `actor_error: interrupted`, and three interruptions in a minute make the store `broken`.
  - `SetMaxCallStackSize` caps recursion.
  - goja has no memory limit, so v3.0 relies on input limits (document size, result rows, response size). Untrusted or memory-hungry apps wait for level 2.
- **Supervisor:** a map from store to actor. It wakes an actor on an event, hibernates it after 60 s idle, and restarts it on panic with backoff. Version swaps work like go-go-goja's `hotreload`: build a candidate runtime, activate (section 6.4), swap, retire the old runtime.
- **Single owner:** each store's writer holds a lease whose epoch is also written into the store's own `_meta.lease_epoch`. Every turn checks that value inside its `BEGIN IMMEDIATE` transaction, so a stale owner (a restarted server, or a level-2 host in v3.1) cannot commit. `system.db` records who holds which lease.

15.4 Server-side sandboxing, in three levels

Level	When	How	What it stops
1. In-process goja	v3.0, trusted authors (the team and its agents)	only kernel modules (no fs, exec or raw network); interrupt budgets; stack cap; authorizer, limits and `max_page_count` on SQL; `fetch` only to allowed origins, after DNS, never to private or loopback addresses, 10 s, 5 MiB; panics recovered per turn	runaway loops, SQL escapes, SSRF, quota abuse, one app reading another's store
2. Actor-host processes	apps from outside the team, or heavy apps	`slop actor-host` child processes, each running the actors of one trust group. A host owns its stores' writer connections under lease, so SQL stays synchronous. It talks to the kernel over a framed JSON protocol on a pipe (inbox events in; commits, outputs and log lines out). Memory and CPU are capped with cgroups v2 or `RLIMIT_AS`; the kernel kills and restarts a host that overruns	memory exhaustion, a stuck VM taking the server down, one tenant's crash affecting others
3. VM per tenant	hostile multi-tenancy, if ever	the level-2 protocol, run inside gVisor or Firecracker	kernel exploits from inside the JS engine

Levels 2 and 3 are v3.1. To keep them possible, nothing in the kernel may assume actors share its process. Actors reach their store only through the turn API, and every message between kernel and actor is serializable.

15.5 Serving and doors

- **HTTP:** `net/http` with a host router (main, site hosts, sandbox wildcard).
  - The views code is ported from v2's `pkg/rt` and rewritten to the new model: a fetch-read event stream with resume, RPC, diff sync.
  - Static serving and image thumbnails come from SLOP-012.
  - Every runtime and shell asset is embedded.
- **Doors:**
  - The verb table generates Glazed commands, so the binary is also the CLI.
  - HTTP handlers live at `/api/v1/<verb>`.
  - MCP runs over stdio and HTTP.
  - Device-code login and labelled tokens come from v2.
- **Models:** `sample` and the docked agent use the geppetto backend with Pi's credentials (SLOP-010), behind francois's `Backend` interface and its fake for tests.
- **Test mode:** `slop serve --test`, on loopback only, adds the virtual clock, fault injection and an `X-Slop-As` header. The conformance runner drives the Go server exactly as it drives the workbench.

15.6 Operations

- **Deployment:** one binary, one data volume (`<root>` of section 4.1). No Postgres.
- **Backups:** nightly `VACUUM INTO` images per changed store, plus `system.db`, plus referenced blobs, copied off-box. Restoring a store means copying its image back while its actor is stopped. Continuous replication per store (Litestream or similar) is a later option, not a v3.0 need.
- **Observability:** per-store counters (turns, CPU, commits, size, errors, reactor lag), the system log, structured logs with store and rev on every line, and `/healthz` and `/readyz`.
- **Upgrades:** the kernel's own schema in each store has a `schema_version` in `_meta`. A new binary migrates stores lazily when it opens them, after taking an image.

15.7 Phases

Phase	Builds	Exit test
G0	ports (SQLite, blobs, clock), `system.db` as the system store, store lifecycle, the verb table and CLI, test mode	the store scenarios pass over HTTP
G1	documents, files, log, snapshots, refs, images, fork, export and import, views and the injected runtime	the contract conformance pages pass in a headless browser; Claude Computer runs
G2	SQL policy, migrations, structured page queries, live queries	every authorizer scenario passes
G3	goja actors: turns, actions, `http`, synchronous reactors, alarms, budgets, supervisor, activation, lease epochs	crash and replay scenarios pass; Blocks folds its log with no tab open
G4	identity, grants, links, guests, approvals, delegates, mounts, sites, the shell	the whole conformance suite is green; v3 serves the team's sites
G5	backups, observability, upgrades	a restore drill from backup; a store moves between two servers as a file
v3.1	actor-host processes with cgroup limits, fork-on-open, durable topics, the rest of section 1's list	a memory-bomb app is killed without affecting others

16. Decision records

Sixteen decisions shape v3. Each row says what was chosen, what was rejected, and what the choice costs.

#	Decision	Rejected	Consequence
D1	The store is the only durable noun; apps, projects, conversations and the system are stores	v2's artifacts with kinds, targets and instances; go-go-host's sites and deployments	one inspector, one history, one export for everything; types and conventions replace tables

#	Decision	Rejected	Consequence
D2	One SQLite file per store, plus `system.db`, which is also the system store	central Postgres (v2, go-go-host); a system store in a separate file	per-store quotas, forks, export and moves are file operations; a platform change and its log entry commit together
D3	A store is an app when its tree has `app.json`	an `app` kind or a separate apps table	software is content: any store can become an app by gaining a manifest, and lose it again
D4	One app per store, with its running version in `_meta`; mounts name only stores	several apps on one store; mounts that pick an app and version	one version, one set of migrations and one origin per store; other apps work through actions or grants; previews run on forks
D5	Server code is `actor.js` with `defineActor`, one actor per store, one event at a time	a request-scoped `server.js`; Express-style apps owning a whole site	Durable Object guarantees with no locks in app code
D6	A turn is one SQLite transaction; an `await` on real I/O ends it; reactors are synchronous; alarms may be async	holding a transaction across awaits; DO-style write coalescing	atomic turns and exactly-once reactors; an async action's continuation can be lost in a crash; work that must finish uses a task row and an alarm
D7	Actors get synchronous SQL; pages get structured queries over exposed tables and views; admins get raw SQL	raw page SQL checked through the authorizer's view name (a CTE can borrow it); documents only	full SQL power for app code; row privacy is a view; pages write through actions

#	Decision	Rejected	Consequence
D8	The per-store log is the event system; events commit with their change; reactor cursors commit with the reactor's writes	a separate pub/sub bus; v2's kernel events table; in-memory rooms only	no lost or doubled effects on stores; no global order across stores
D9	Access = level on the store against a rule, plus capabilities of the app version; one `can()` for every caller	v2's nine mechanisms	guests, links, tokens and delegates are only subjects or caps; every denial has one explanation
D10	Mounts only map host and path to a store	route tables that bind data, records and page choice	routing logic is app code; a "site" app replaces `record` and `page_from`
D11	Pages keep the claude.ai contract exactly; slop's extras live under `claude.use('slop')`	a slop-specific page API	Computer, Blocks and Pinup run unchanged; slop features never collide with future contract names
D12	One verb table generates CLI, HTTP, MCP and the agent's `eval` scope	hand-written commands per door (francois's 67 MCP tools)	a feature is one verb; agents and people have the same powers under the same checks
D13	Two kinds of surface: iframe apps and `slop.ui.v1` widget trees; the shell is itself an app	React modules compiled into the shell (go-go-os)	apps extend the UI safely; the widget pack stays small on purpose
D14	Build the browser workbench first, behind six ports, with a shared conformance suite	Go first; a spec in prose only	the design is tested before Go code exists; the Go port has an executable target
D15	No backwards compatibility with v2	importers and adapters	v2 data moves, if at all, as exported folders imported as stores; no v2 code paths survive

#	Decision	Rejected	Consequence
D16	v3.0 ships the core; the list in section 1 is designed but deferred to v3.1	everything at once	a usable system sooner; v3.0 choices must not block the deferred parts

17. Open questions and risks

The biggest risk is that the turn boundary at `await` surprises app authors. The workbench exists partly to find that out early. The rest are known costs, each with a mitigation and the phase where it gets tested.

Risk	Why it matters	Mitigation	Tested in
Writes before an `await` commit on their own	an action that half-runs leaves half its writes	the turn inspector shows each commit; `ctx.exclusive`; a lint warning when a handler writes, awaits, then writes again; the task-row-plus-alarm pattern in the book	W3
Async continuations are lost in a crash	a workflow stops halfway and its caller gets `actor_error: lost`	anything that must finish is a task row plus an alarm, which retries	W3
Row privacy depends on views the app writes	a mistake exposes rows	nothing is exposed by default; pages can name only exposed tables and views; the linter warns when a table with an `owner` column is exposed directly; the `can()` explainer shows what allowed a read	W2, W5

Risk	Why it matters	Mitigation	Tested in
goja has no memory limit	one app can exhaust the server in v3.0	input and result limits; only the team's apps on v3.0; actor-host processes in v3.1	G3
QuickJS and goja differ	an actor passes in the workbench and fails in Go	the dialect lint; the conformance suite runs every example actor on both engines	W3, G3
`opfs-sahpool` is single-tab and grows asynchronously	a second tab, or a full pool, stalls the workbench	a Web Lock at start; 16 spare file slots, refilled between turns	W0
Many store files	file descriptors and memory at 10,000 stores	an LRU of 512 open stores; hibernated actors release their connections	G5
Live queries re-run often	CPU on busy tables	read sets limit which commits trigger a re-run; 100 ms throttle; 32 watches per view; 250 ms budget	W2
The claude.ai contract moves	hosted apps break on new contract versions	the runtime records the contract version it implements; the contract conformance pages run in CI	W4, G1
Images (`VACUUM INTO`) of large stores are slow	a snapshot or fork delays the next turn	images run between turns, on demand, before activation and daily; a 256 MiB default store limit	W1, G5
The workbench's fidelity gaps hide bugs	no WAL, simulated origins, same-origin iframes	gaps are labelled in the UI; Go runs the suite in a real browser with real origins	G1

Risk	Why it matters	Mitigation	Tested in
One app per store limits composition	Blocks and a lens both want to write one workspace	reads and actions across stores by grant; revisit if a real case cannot be served	W4

Open questions

- [ ] Should fork-on-open be allowed on public mounts by default? Proposed: no; an admin turns it on per app.
- [ ] Do guests ever get `sample`? Proposed: no, unless the app's owner pays through a per-app budget (out of scope for v3.0).
- [ ] Should admins be able to run DDL from the Data surface while prototyping? Proposed: yes, and each statement is saved as a new migration file in the app's tree, so the store and the code never disagree.
- [ ] Where do secrets live? Proposed: in `system.db`, encrypted with a key from the server's environment; readable only through `use('secrets')` by approved app versions.
- [ ] How are level-2 actor hosts grouped: per app, per owner or per trust level? Proposed: per owner.
- [ ] What does a `project` store hold? Proposed: `BRIEF.md`, a `stores` document listing members with roles (page, content, samples, system), and its agent conversations by reference.
- [ ] Is `actor.js` the final name? Proposed: yes. It says what the code is (the store's actor), and `server.js` suggests a request-scoped server that this is not.

18. Appendices

The schemas and references below are the starting point for W0 to W3. They are exact enough to code against; field names may still change during the workbench.

A. `system.db`

```
-- system.db is also the system store: it holds _meta, _docs, _log and
_changes exactly as in Appendix B,
-- so a registry change and its log entry commit together. Questions are
documents under questions/.
CREATE TABLE stores (
  id TEXT PRIMARY KEY, type TEXT NOT NULL DEFAULT '', app TEXT, owner TEXT
NOT NULL,
  created_at INTEGER NOT NULL, forked_from TEXT, made_by TEXT, deleted_at
INTEGER,
  size_bytes INTEGER NOT NULL DEFAULT 0);
CREATE TABLE users (
  id TEXT PRIMARY KEY, issuer TEXT NOT NULL, subject TEXT NOT NULL, email
TEXT NOT NULL DEFAULT '',
  name TEXT NOT NULL DEFAULT '', role TEXT NOT NULL DEFAULT 'pending'
    CHECK (role IN ('admin','member','pending','disabled')),
  created_at INTEGER NOT NULL, last_seen_at INTEGER, UNIQUE (issuer,
subject));
CREATE TABLE visitors (id TEXT PRIMARY KEY, created_at INTEGER NOT NULL,
last_seen_at INTEGER NOT NULL,
  claimed_by TEXT REFERENCES users(id));
CREATE TABLE sessions (token_hash BLOB PRIMARY KEY, host TEXT NOT NULL,
principal TEXT NOT NULL, csrf TEXT NOT NULL,
  created_at INTEGER NOT NULL, last_seen_at INTEGER NOT NULL, expires_at
INTEGER NOT NULL,
  absolute_expires_at INTEGER NOT NULL, revoked_at INTEGER, from_visitor
TEXT);
-- login_flows, site_handoffs, device_authorizations: as in v2 (SLOP-011,
SLOP-013), timestamps as INTEGER ms
CREATE TABLE tokens (id TEXT PRIMARY KEY, user_id TEXT NOT NULL REFERENCES
users(id), name TEXT NOT NULL,
  token_hash BLOB NOT NULL UNIQUE, max_level TEXT NOT NULL, stores TEXT NOT
NULL DEFAULT '*',
  created_at INTEGER NOT NULL, last_used_at INTEGER, expires_at INTEGER,
revoked_at INTEGER);
CREATE TABLE grants (store TEXT NOT NULL REFERENCES stores(id), subject TEXT
NOT NULL,
  level TEXT NOT NULL CHECK (level IN ('view','interact','admin','owner')),
via TEXT NOT NULL DEFAULT 'direct',
  created_by TEXT, created_at INTEGER NOT NULL, expires_at INTEGER, PRIMARY
KEY (store, subject, via));
CREATE TABLE instantiate (app TEXT NOT NULL REFERENCES stores(id), subject
```

```
TEXT NOT NULL,
  template TEXT REFERENCES stores(id), per_day INTEGER NOT NULL DEFAULT 50,
  created_by TEXT, created_at INTEGER NOT NULL, PRIMARY KEY (app, subject));
CREATE TABLE links (id TEXT PRIMARY KEY, store TEXT NOT NULL REFERENCES
stores(id), token_hash BLOB NOT NULL UNIQUE,
  level TEXT NOT NULL CHECK (level IN ('view','interact')), created_by TEXT,
created_at INTEGER NOT NULL,
  expires_at INTEGER, revoked_at INTEGER);
CREATE TABLE delegates (id TEXT PRIMARY KEY, token_hash BLOB NOT NULL UNIQUE,
parent TEXT NOT NULL,
  user_id TEXT NOT NULL, store TEXT NOT NULL, scope TEXT NOT NULL DEFAULT '',
caps TEXT NOT NULL, level TEXT NOT NULL,
  created_at INTEGER NOT NULL, expires_at INTEGER NOT NULL, revoked_at
INTEGER);
CREATE TABLE approvals (app TEXT NOT NULL, version INTEGER NOT NULL, caps
TEXT NOT NULL,
  approved_by TEXT NOT NULL, approved_at INTEGER NOT NULL, PRIMARY KEY (app,
version));
CREATE TABLE consents (user_id TEXT NOT NULL, app TEXT NOT NULL, capability
TEXT NOT NULL,
  state TEXT NOT NULL CHECK (state IN ('granted','denied')), at INTEGER NOT
NULL,
  PRIMARY KEY (user_id, app, capability));
-- mounts: section 10.2
CREATE TABLE schedules (store TEXT PRIMARY KEY REFERENCES stores(id), next_at
INTEGER NOT NULL);
CREATE INDEX schedules_next ON schedules (next_at);
CREATE TABLE steps (id TEXT PRIMARY KEY, principal TEXT NOT NULL, via TEXT
NOT NULL, message TEXT NOT NULL DEFAULT '',
  project TEXT, started_at INTEGER NOT NULL, ended_at INTEGER,
  state TEXT NOT NULL DEFAULT 'open' CHECK (state IN
('open','done','kept','reverted')), reverted_by TEXT);
CREATE TABLE step_commits (step TEXT NOT NULL REFERENCES steps(id), store
TEXT NOT NULL, epoch INTEGER NOT NULL,
  rev INTEGER NOT NULL, PRIMARY KEY (step, store, epoch, rev));
CREATE TABLE leases (store TEXT PRIMARY KEY, holder TEXT NOT NULL, epoch
INTEGER NOT NULL, until INTEGER NOT NULL);
CREATE TABLE blob_refs (sha256 TEXT PRIMARY KEY, refs INTEGER NOT NULL,
zero_since INTEGER);
CREATE TABLE secrets (app TEXT NOT NULL, name TEXT NOT NULL, value BLOB NOT
NULL, set_by TEXT, set_at INTEGER NOT NULL,
  PRIMARY KEY (app, name));
CREATE TABLE session_forks (session TEXT NOT NULL, host TEXT NOT NULL, path
```

```
TEXT NOT NULL, store TEXT NOT NULL,
  PRIMARY KEY (session, host, path));                  -- fork-on-open, v3.1
CREATE TABLE server_keys (name TEXT PRIMARY KEY, value BLOB NOT NULL,
created_at INTEGER NOT NULL);
```

B. `store.db`

```
CREATE TABLE _meta (key TEXT PRIMARY KEY, value TEXT NOT NULL);
  -- id, type, app, app_ref, app_pin, app_version, last_good, owner,
created_at, forked_from, forked_at,
  -- made_by, epoch, lease_epoch, schema_version, limits (JSON)
CREATE TABLE _docs (path TEXT PRIMARY KEY, coll TEXT NOT NULL,
  body TEXT CHECK (body IS NULL OR json_valid(body)), version INTEGER NOT
NULL, rev INTEGER NOT NULL,
  size INTEGER NOT NULL, by TEXT, at INTEGER NOT NULL) WITHOUT ROWID;
CREATE INDEX _docs_coll ON _docs (coll);
CREATE INDEX _docs_rev ON _docs (rev);
CREATE TABLE _files (zone TEXT NOT NULL CHECK (zone IN ('tree','assets')),
path TEXT NOT NULL, sha256 TEXT,
  size INTEGER NOT NULL, type TEXT NOT NULL, rev INTEGER NOT NULL, by TEXT,
at INTEGER NOT NULL,
  PRIMARY KEY (zone, path)) WITHOUT ROWID;
CREATE TABLE _log (rev INTEGER PRIMARY KEY, at INTEGER NOT NULL, principal
TEXT NOT NULL, via TEXT NOT NULL,
  app TEXT, step TEXT, summary TEXT NOT NULL DEFAULT '');
CREATE TABLE _changes (rev INTEGER NOT NULL, seq INTEGER NOT NULL, kind TEXT
NOT NULL, key TEXT NOT NULL,
  op TEXT NOT NULL, version INTEGER, old TEXT, new TEXT, PRIMARY KEY (rev,
seq)) WITHOUT ROWID;
CREATE INDEX _changes_key ON _changes (kind, key, rev);
CREATE TABLE _snapshots (n INTEGER PRIMARY KEY, rev INTEGER NOT NULL, tree
TEXT NOT NULL, image TEXT,
  message TEXT NOT NULL DEFAULT '', by TEXT, at INTEGER NOT NULL, parent
INTEGER);
CREATE TABLE _snapshot_files (n INTEGER NOT NULL, path TEXT NOT NULL, sha256
TEXT NOT NULL, size INTEGER NOT NULL,
  type TEXT NOT NULL, PRIMARY KEY (n, path)) WITHOUT ROWID;
CREATE TABLE _refs (name TEXT PRIMARY KEY, n INTEGER NOT NULL REFERENCES
_snapshots(n), at INTEGER NOT NULL, by TEXT);
CREATE TABLE _migrations (app TEXT NOT NULL, name TEXT NOT NULL, sha256 TEXT
NOT NULL, rev INTEGER NOT NULL,
  PRIMARY KEY (app, name));
```

```
CREATE TABLE _cursors (name TEXT NOT NULL, source TEXT NOT NULL, epoch
INTEGER NOT NULL, rev INTEGER NOT NULL,
  PRIMARY KEY (name, source));
CREATE TABLE _alarms (name TEXT PRIMARY KEY, at INTEGER NOT NULL, every TEXT,
data TEXT,
  running INTEGER NOT NULL DEFAULT 0);
CREATE TABLE _leases (path TEXT PRIMARY KEY, holder TEXT NOT NULL, until
INTEGER NOT NULL, data TEXT);
```

C. `app.json` fields

Field	Type	Default	Meaning
`title`	string	the store id	display name
`page`	path	none	HTML entry; absent = no page
`actor`	path	none	actor entry; absent = no actor
`opens`	string	`""`	the store type this app runs on and creates
`capabilities`	object	`{}`	the claude.ai contract block: `db` (with `rules`, `indexes`), `user`, `room` (`topics` maps a topic to a level), `comments`, `assets`, `downloads`, `artifact`, `sample`, `mcp`
`slop.sql.expose`	map	`{}`	table or view to `{ read: level }`
`slop.fetch.allow`	origins	`[]`	origins `fetch` may reach
`slop.secrets`	names	`[]`	this app's secrets the actor may read
`slop.spawn`	bool	`false`	may create child views and one-level delegates
`slop.platform`	permissions	`[]`	`stores:create`, `mounts:request`, `mounts:write`, `grants:write`, `refs:write`, `ask`
`slop.surfaces`	list	`[]`	`{ id, title, slot, render }`; `slot` is `surface` or `opener:<type>`; `render` is `actor:<name>` or a page path

Field	Type	Default	Meaning
`slop.api`	path	none	Markdown describing actions, read by agents
(not declared)			always on: `sql`, `files`, `log`, `alarm`, `time`, `ui`, `actions`, `stores.open`, `permissions`, `user`, and `claude.use('slop')` in pages

D. Actor reference

```
defineActor({
  setup?(ctx): void                                    // in the activation
transaction
  smoke?(ctx): void                                    // in the activation
transaction; throw to fail it
  actions?: { [name]: (ctx, args: Json) => Json | Promise<Json> }
  http?:    { ['METHOD /path/:param']: { level: Level | 'public', run: (ctx,
req: Req) => Res | Promise<Res> } }
  on?:      { [selector]: (ctx, commit: Commit) => void }          //
synchronous; selectors in section 8.1
            // 'channel:<topic>' handlers get (ctx, msg: { topic, data, peer,
at })
  every?:   { [name]: { cron: string, run: (ctx) => void | Promise<void> } }
// may be async; at least once
  surfaces?:{ [name]: (ctx, props: Json) => UiNode }
})

ctx = { principal: { id, kind, name, level }, store: { id, type }, app: { id,
version },
        now: number, rev: number, step: string | null,
        event: { kind: 'action'|'http'|'change'|'channel'|'alarm'|'setup',
late?: boolean, onBehalfOf?: string },
        need(level): void, exclusive<T>(fn: () => Promise<T>): Promise<T>,
log(...args): void }

Commit = { store, epoch, rev, at, principal, via, app, step, changes:
Change[] }
Change = { kind, key, op, version? }
```

```
Req = { method, path, params, query, headers, body: Json | string | null, ip
}   // principal 'anon' on public routes
Res = { status?: number, headers?: object, json?: Json, text?: string,
bytes?: Uint8Array, html?: string }
```

E. Wire protocol

View RPC methods (`POST /api/views/<v>/rpc {method, args}`):

Namespace	Methods
`db`	`get` , `set` , `update` , `delete` , `query` , `subscribe` , `unsubscribe` , `acquire` , `release`
`room`	`join` , `leave` , `presence` , `emit`
`assets`	`upload` , `list` , `delete`
`comments`	`create` , `reply` , `resolve` , `delete` , `list`
`sample`	`create` , `cancel` , `toolResult`
`artifact`	`publish`
`slop.sql`	`select` , `watch` , `unwatch`
`slop.files`	`read` , `list`
`slop.log`	`watch` , `since`
`slop.actions`	`call`
`slop.spawn`	`view`
`slop.platform`	one method per permission

View stream events (`GET /api/views/<v>/events?after=<rev>`):

- `hello {view, level, caps, me, epoch, rev}` ;
- `commit {epoch, rev, changes}` ;
- `rows {watch, rows}` ;
- `peers {...}` and `room {...}` ;
- `sample {call, delta}` , `tool_use {call, use, tool, input}` and `sample_end {call, result | error}` ;

- `reload {version}`;
- `revoked {}`;
- `reset {epoch, rev}`.

The stream is read with fetch streaming, not `EventSource`, so it can carry the `Authorization` header.

Workbench internal messages:

- **Shell to kernel,** over the shell's own `MessagePort`: `{id, as, verb, args}`, answered with `{id, result}` or `{id, error: {code, message}}`. The kernel pushes `{sub, event}`. Only this port accepts `as`.
- **Page to kernel,** over one `MessagePort` per view: the view RPC methods and stream events above, bound to that view's principal.
- **Service worker to kernel,** over a `MessagePort` the shell transfers at start: `{id, kind: 'http', host, path, method, headers, body}`, answered with `{id, status, headers, body}`.

F. A worked example: a journal

```
journal/  (one store: it is the app and it holds the posts)
  app.json        { "title": "Journal", "page": "index.html", "actor":
"actor.js",
                    "capabilities": { "user": {} },
                    "slop": { "sql": { "expose": { "published": { "read":
"view" } } }, "api": "API.md" } }
  sql/001_posts.sql
                  CREATE TABLE posts (id INTEGER PRIMARY KEY, slug TEXT
UNIQUE, title TEXT, body TEXT,
                                      draft INTEGER NOT NULL DEFAULT 1, at
INTEGER);
                  CREATE VIEW published AS SELECT slug, title, body, at FROM
posts WHERE draft = 0;
  actor.js        const sql = use('sql');
                  defineActor({
                    actions: {
                      draft(ctx, { slug, title, body }) { ctx.need('admin');
                        sql.run('INSERT INTO posts(slug, title, body) VALUES
(?,?,?)', slug, title, body); },
                      publish(ctx, { slug }) { ctx.need('admin');
                        sql.run('UPDATE posts SET draft = 0, at = ? WHERE
slug = ?', ctx.now, slug); } },
                    http: { 'GET /feed.json': { level: 'public',
                              run: () => ({ json: sql.all('SELECT slug,
title, at FROM published ORDER BY at DESC LIMIT 20') }) } },
                  });
  index.html      lists posts with slop.sql.watch('published', { orderBy:
[['at', 'desc']] }, render)
```

```
slop push ./journal --store journal --create -m "journal app"   # create,
write the tree, snapshot, activate (migrations run)
slop call journal draft --slug hello --title Hello --body "First post"
slop call journal publish --slug hello                           # actor
actions, admin only
slop mount set site.slop.computer /journal journal
curl https://site.slop.computer/journal/feed.json                # actor
http, public
```

Every line above is a commit in `journal`'s log or the system log:

- the push is one commit plus snapshot 1, and activation is another;

- the draft and the publish are actor turns, each recording a `table` change on `posts` with a `row` hint;
- the mount is a commit in the system log.

The Activity surface shows them all in one step if they ran under `slop step start`.

Sources

Code and documents read for this design:

- go-go-golems/slop-computer: code, migrations 001 to 012, tickets SLOP-001 to SLOP-013, and the 28 October directions brainstorm.
- go-go-golems/francois.
- go-go-golems/go-go-host.
- go-go-golems/go-go-goja.
- go-go-golems/go-go-os-frontend, go-go-os-backend and go-go-os-chat.
- HazAT/ultimate and the Pi Durable post.
- The essay Software is just content.
- Attached: Claude Computer 1.0.0, Claude Blocks, Meta Blocks 2, RM_HYPHA (Librarian), the Pinup export, the three Form Skate design systems and the executable-textbook handbook.

Engine facts this design relies on:

- The SQLite WASM build documents `sqlite3_set_authorizer()`, the hook APIs and the session and changeset APIs in its C-style API: sqlite.org/wasm api-c-style.
- The `opfs-sahpool` VFS needs no extra worker and notes that WAL is unavailable in the WASM build: VFS source.
- `mattn/go-sqlite3` exposes `RegisterAuthorizer`, `RegisterUpdateHook`, `RegisterCommitHook`, `RegisterFunc`, `Backup` and `Deserialize`: pkg.go.dev. The authorizer is called only while a statement is prepared: sqlite.org.
- goja has had async/await since December 2022 (PR #464); its ES module support is still in progress (issue #498).
- quickjs-emscripten runtimes offer `setInterruptHandler`, `setMemoryLimit` and `setMaxStackSize`: runtime docs.